

- Syllabus:
1. Lexical Analysis
 2. Parsing
 3. Syntax Directed Translation (SDT)
 4. Intermediate code generation
 5. Basics of code optimization

Translator:

A software system which convert the source code from one form of language to another form of language is called Translator.

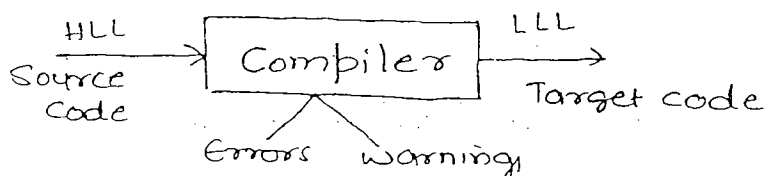
Types of Translators.

There are two type of Translators

1. Compiler
2. Assembler.

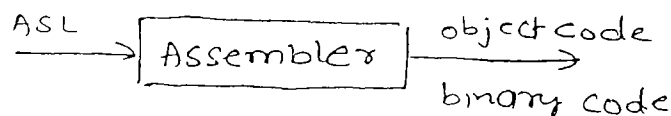
⇒ Compiler:

A translator which convert the source code from high level language to low level language is called compiler.

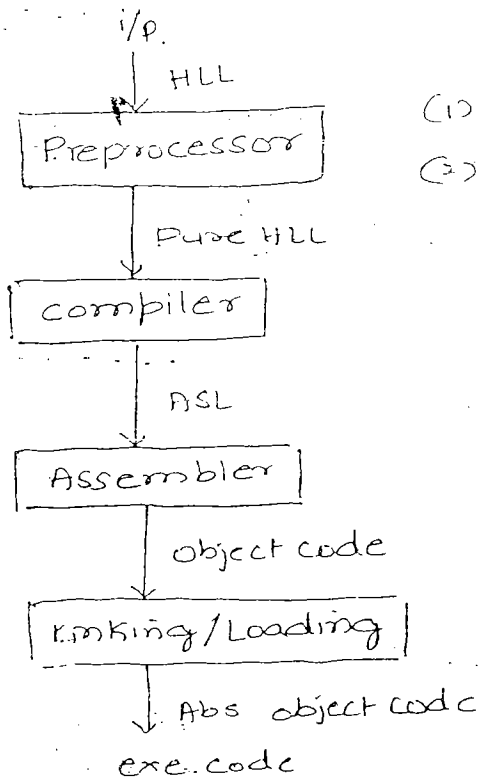


⇒ Assembler:

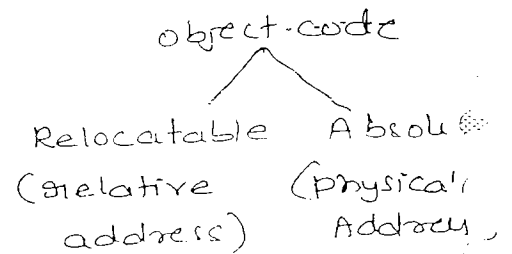
A software s/m which converts assembly code into object code or Binary code is called Assembler



Functions of language processing system (L.Ps):



- (1) File Inclusion
- (2) MACRO Evaluation



- 1) Preprocessor includes all the header files and also evaluates if any macro is included.
The preprocessor is also called a MACRO Evaluator. Preprocessing is optional i.e. if any language which does not support #include and MACRO's preprocessing is not required.
- 2) Compiler takes the preprocessor as input and converts it into assembly code.
- 3) Assembler converts the assembly language into object code or (binary code) or (machine code).
- 4) Linking and Loading provides four functions.
 - a. Allocation
 - b. Relocation
 - c. Linker
 - d. Loader.

Allocation :
Getting the memory portions from operating system and storing the object data or object code.

Relocation:

mapping the relative address to the physical address and relocating the object code. 2

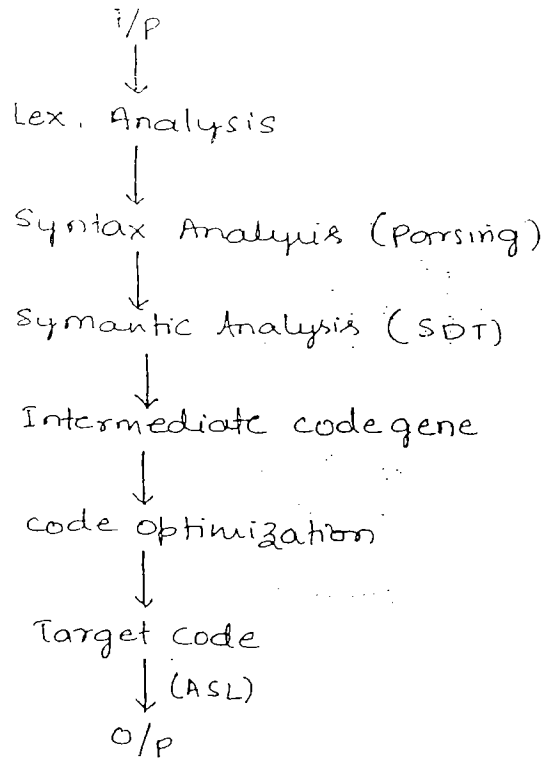
Linker:

Combines all executable object module to produce single executable file.

Loader:

Loading the executable file into permanent memory.

Design of Compiler:



Phase:

Converting the source code from one form of representation to another form of representation is called Phase.

Lexical Analysis:

Lexical Analyser scan the source code and divide into tokens. i.e input is source code and o/p is stream of tokens.

Ex: int | x | = | 20 | ;

↓ ↓ ↓ ↓ ↓

kw id op const op

kw id op const op

2. Syntax Analysis:

Syntax analyzer verify the grammatical mistake of the source code. To verify the syntax of the source code the language must be defined by CFG. Syntax analyzer take the stream of tokens as i/p & generate the parse tree.

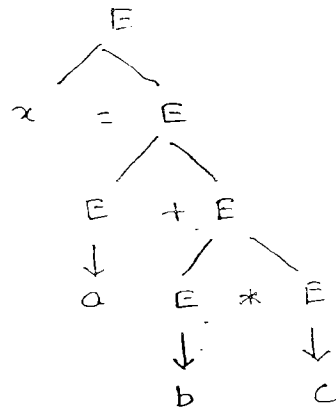
Ex: $x = a + b * c$

CFG

$E \rightarrow id$

$E \rightarrow E + E \mid E * E \mid id$

Parse Tree:



3. Semantic Analysis:

Semantic Analysis verify the meaning of each and every sentence by performing type check type

Syntax analyzer just verifies whether the operator is operating on required no. of operands or not and does not look into working

Ex: `int x;`

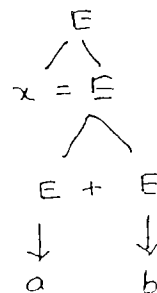
`char a;`

`float b;`

`x = a + b` { No Syntax Errors
Semantic Errors

$E \rightarrow id = E$

$E \rightarrow E + E \mid id$



4. Intermediate code generation:

The source code is converted into intermediate representation to make code generation process simple & easy & also to achieve the platform independency.

Ex: $x = a + b * c$

$t_1 = b * c$

$t_2 = a + t_1$

$x = t_2$

} Intermediate code.

5. Code optimization.

Reducing the number of instructions without affecting the outcome of the source program is called code optimization.

Optimization is of two types.

1. Machine independent optimization
2. Machine dependent optimization.

Machine independent optimization:

$x = a + b$; $\left\{ \begin{array}{l} \text{Load} \\ \text{ADD} \\ \text{move} \end{array} \right.$

$y = b + a$; $\left\{ \begin{array}{l} L \\ A \\ m \end{array} \right.$

$\Rightarrow x = a + b;$

$y = x;$

6. Target code:

The optimized source code should be converted into assembly code.

Ex: $x = a + b * c;$

MUL R₁ R₂ // $b * c$

ADD R₀ R₂ // $a + b * c$

move R₂ x // $x = a + b * c.$

R₀ $\rightarrow$ a

R₁ $\rightarrow$ b

R₂ $\rightarrow$ c.

Note:

Code optimization phase divides the compiler into two

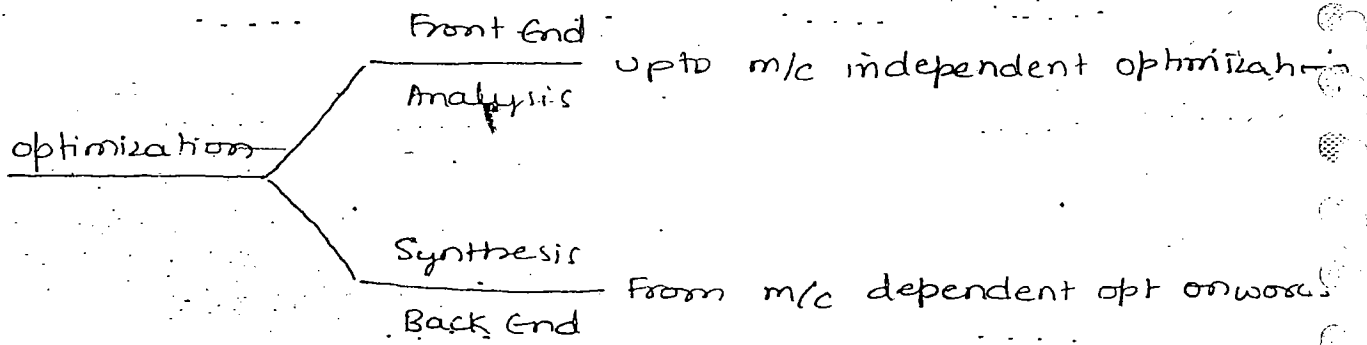
Parts

(a) Front End (Analysis)

This document is available free of charge on

 **studocu**

Downloaded by Personal Work (personalwork671@gmail.com)



Interpreter:

Software system that convert the source code into executable code is an Interpreter.

Compiler

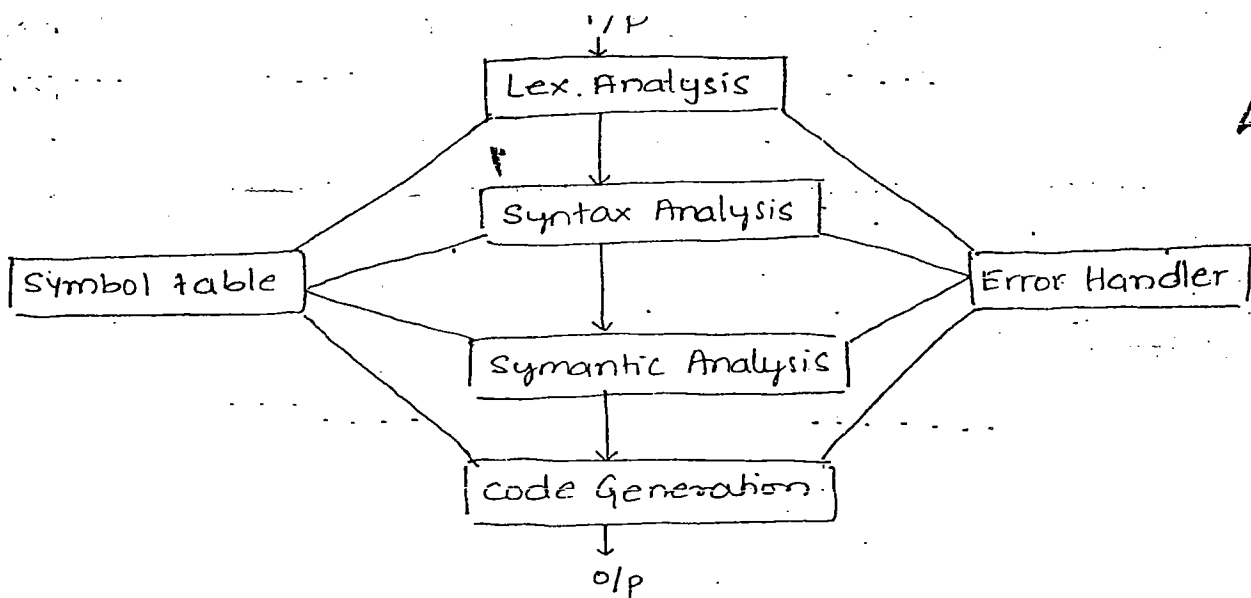
- I/p: HLL
O/p: LLL
i.e process is Translation
- Scan the entire
- Requires ~~less~~ ^{more} memory but time is less.
- code optimization is possible
- Scope is static
- FORTAN is the first language

Interpreter

- I/p: source code
O/p: result.
Process is execution.
- scan the source code line by line.
- Requires less memory but time is more.
- code optimization is not possible.
- scope is dynamic

Note:

Some languages are both compiler and interpreter orientation. Ex: Java.



Symbol table :

Symbol table ADT used by the compiler to store the complete information of source code.

At any phase if any variable is encountered that will be checked in the symbol table.

During each and every phase of compiler.

Lexical Analysis is the first phase interact with Symbol table and symbol table is created by the Lexical Analysis.

Compiler will provide the memory for symbol table. Information of the identifier stored in the symbol table:

1. Name
2. Type, size
3. Location
4. scope
5. other information like array size for arrays
- 6.

Symbol table can be implemented by any one of the following data structure

1. Linear Table
2. List
3. Tree
4. Hash Table

operation on Symbol table

1. Insert
2. Look up
3. modify
4. Delete.

Error Handler:

Error handler apart from collecting the errors from different phases of and also responsible for continuing the compiler even error occur.

Compiler can handle three types of error.

① Lexical Error

ex: int = "Hello";

int i = 10;

(i) = 20;

② Syntax Error

ex: 1. int x = 10;

2. {
}

3. (x) if
{
}

③ Semantic Errors

ex: x = a + b;

↓ ↓ ↓
int char boolean

Apart from these three errors we may have fatal errors which are generated by systems.

Phase(1) (Lexical Analysis):

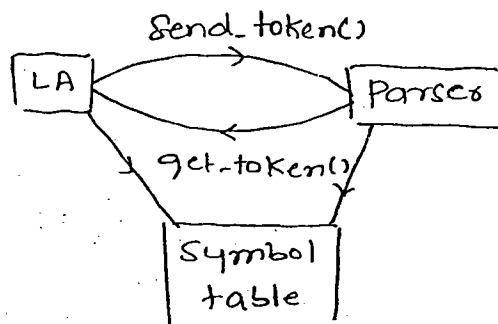
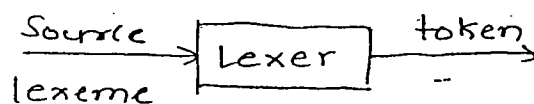
Lexical Analyzer is also called as Lexer. Lexer read the source code and divide the source code into tokens. Where token is a group of characters with logical meaning.

Lexeme: is the actual representation of stream of character

Ex: 1. $\pi = 3.1415$

↓
id

The lexer take the stream of lexeme and o/p stream of tokens.



Construction of lexical Analyzer.

1. Define the rule based on the i/p stream these rule are pattern recognizing tools.
2. Construct the Regular Expression
3. Convert the RE into Finite Automata.

They are two ways for the construct of lexer

- ① Hand Code
- ② Lex tool.

Lexical Analyzer is also called as ^{or} scanner or tokenizer.

Find the no. of tokens in the following source code.

1. $\text{printf} \left(\underset{1}{C} \left| \underset{2}{\text{"}} \underset{3}{\text{TOC "}} \underset{4}{\text{)}} \underset{5}{\text{;}} \right| \right)$ No. of tokens = 5

$$2. \quad \frac{DO}{1} = \frac{I}{2} = \frac{15.5}{4} = \frac{3.875}{5}$$

3.

```
int  Add (int x, int y)
{
    return x + y;
}
```

4. printf (" i = %.d , \$i = %.9 " , i , &i);
 1 2 3 4 5 6 7 8 9 10

Secondary function of lexical Analyzer:

1. Remove the comment lines and whitespace characters.
2. Correlating the error messages by tracking the line no.

PARSING:

The process of constructing the parse tree for a given input string is called Parsing.

To parse any i/p string, the language must be defined by context free grammar (CFG).

Grammar:

Finite set of rules that may generate infinite no. of sentences is called grammar.

⑥

Grammar is a 4 tuple $G = (V, T, P, S)$

$\gamma =$ set of all Non-terminals

$T =$ set of all terminals

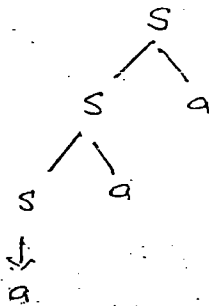
$P =$ set of all productions

S - start symbol

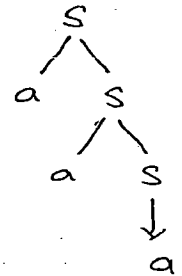
Ex: $S \rightarrow aS \mid Sa \mid a$

$w = aaa$

LMD: $S \rightarrow Sa$
 $\rightarrow Saa$
 $\rightarrow aaa$



RMD: $S \rightarrow aS$
 $\rightarrow aaaS$
 $\rightarrow aaa$



Classification of the Grammar:

1. Based on the No. of derivation trees (Classification-I)
2. Based on the No. of strings (Classification-II)

Classification-I

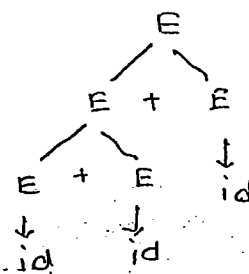
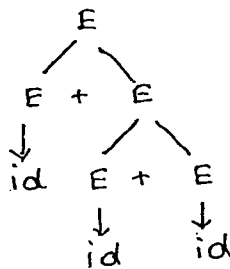
1. Ambiguous Grammar
2. Unambiguous Grammar

Ambiguous Grammar:

The Grammar 'G' is said to be Ambiguous if there exist more than one derivation tree for the given i/p string more than one LMDT or RMDT.

Ex: $E \rightarrow E + E \mid id$

$w = id + id + id$



$\Rightarrow$ The grammar G is Ambiguous

NOTE:

The grammar whose productions are in the form 6

$A \rightarrow \alpha, A \in V, \alpha \in (V \cup T)^*$ is called CFG.

Ex: ① $E \rightarrow E + E \mid E * E \mid id$

$\Sigma = \{+, *, id\}$

② $E \rightarrow E + E$

$\rightarrow (E)$

$\rightarrow E == E$

$\rightarrow id$

$T = \{+, ==, (,), id\}$

③ $S \rightarrow A \wedge B + C (D)$

$A \rightarrow id$

$B \rightarrow id$

$C \rightarrow id$

$D \rightarrow id$

$\Sigma = \{\wedge, +, (,), id\}$

Derivation:

The process of deriving a string is called derivation and geometrical representation is called Derivation tree/parse tree/syntax tree.

All the intermediate steps in the derivation is called sentential form.

$E \rightarrow E + E$

$\rightarrow E + E * E$

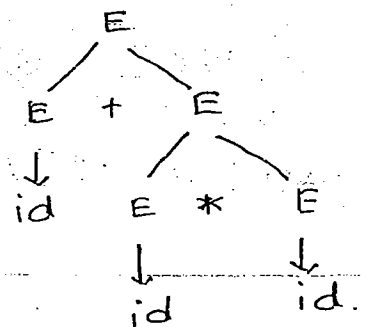
$\rightarrow id + E * E$

$\rightarrow id + id * E$

$\rightarrow id + id * id$

it is sentential form

Parse Tree



Types of derivation:

1. LMD (Left most derivation)

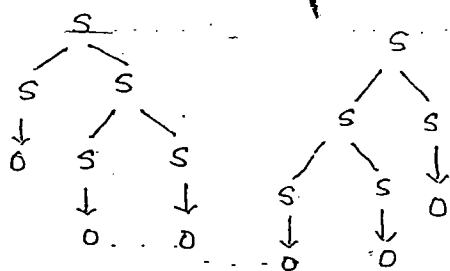
2. RMD (Right most derivation)

LMD: The process of deriving a string by expanding the left most non terminal is called LMD and geometrical representation of LMD is called LMDT.

RMD: The process of deriving a string by expanding right most variable is called RMD and geometrical representation of RMD is called RMDT.

$$2. \quad S \rightarrow ss|0$$

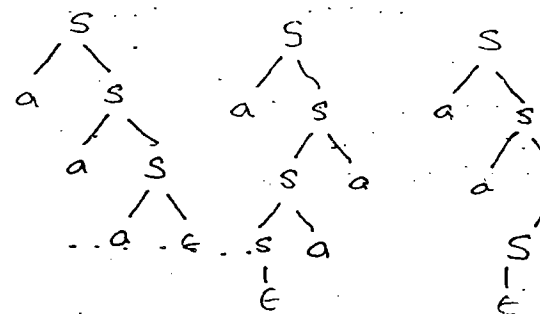
$$w = 000$$



$$3. \quad S \rightarrow as|Sa|\epsilon$$

$$w = aaa$$

7



No. of derivation = 6.

Note:

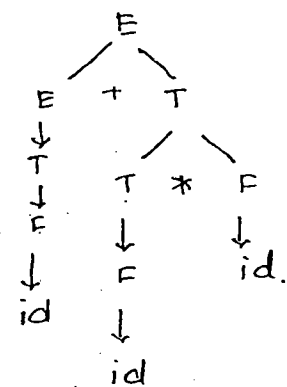
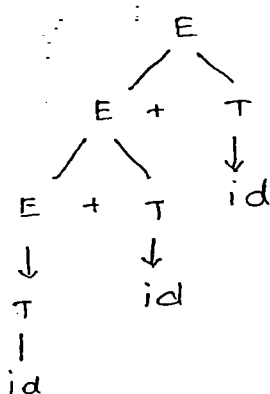
1. In ambiguous Grammar both LMD & RMD represent different parse tree $LMDT \neq RMDT$
2. The ambiguity of CFG is Undecidable.
3. Ambiguity can be eliminated by rewriting the Grammar.

Ex: $E \rightarrow E + E / id$

2. $E \rightarrow E + E | E * E | id$ AG.

$$\left. \begin{array}{l} E \rightarrow E + T / T \\ T \rightarrow id \\ w = id + id + id \end{array} \right\} \text{UAG.}$$

$$\left. \begin{array}{l} E \rightarrow E + T / T \\ T \rightarrow T * F / F \\ F \rightarrow id \end{array} \right\} \text{UAG.}$$



Unambiguous Grammar:

The grammar 'G' is said to be Unambiguous if $\exists$ unique derivation tree for every i/p string.

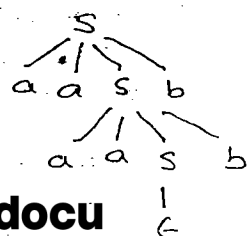
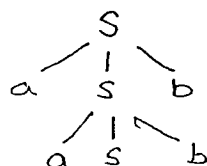
Ex: $S \rightarrow asb|\epsilon$

②

$$S \rightarrow aasb|\epsilon$$

$$w = aabb \quad \text{UAG}$$

$$w = a a a b b \quad \text{UAG.}$$



Verifying the following grammar is ambiguous or unambiguous

1. $S \rightarrow A|a$
 $A \rightarrow a$ (A)
2. $S \rightarrow SaS|b$ (A)
3. $S \rightarrow SSS|a$ (A)
4. $S \rightarrow Sasb|sbSa|\epsilon$ (A)
5. $A \rightarrow A(A)|a$ (A)
6. $A \rightarrow AA|(A)|a$ (A)
7. $S \rightarrow aSbS|\epsilon$ (A)
8. $S \rightarrow SS|AB$
 $A \rightarrow Aa|a$
 $B \rightarrow Bb|b$ (A)
9. $S \rightarrow (L)|a$ (UA)
 $L \rightarrow L, S|S$
10. $S \rightarrow AA$
 $A \rightarrow aA$ (UA)
 $A \rightarrow b$

Classification of Grammar:

- ① Recursive Grammar
- ② Non-Recursive Grammar

Recursive Grammar

The grammar 'G' is said to be recursive if $\exists$ at least one production which contains same variable both at LHS and RHS.

Ex: ① $S \rightarrow Sa|b$ ② $S \rightarrow as|b$ ③ $S \rightarrow asb|\epsilon$

Non recursive Grammar

The grammar 'G' is said to be non recursive if no production contains same variable both at LHS & RHS

Ex: ① $S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$ ② $S \rightarrow A|OB$
 $A \rightarrow 0|1$
 $B \rightarrow 1|C$
 $C \rightarrow 0|1$

Note:

1. The grammar 'G' is recursive iff it generates infinite language
2. The grammar 'G' is non-recursive iff it generates finite language

Types of Recursion:

① Left Recursion

② Right Recursion

Left Recursion:

The grammar is said to be left recursive if the left most variable of RHS is same as the variable at LHS.

ex: 1. $S \rightarrow Sa | b$

2. $S \rightarrow AB$
 $A \rightarrow Aa | b$
 $B \rightarrow aB | a$

3. $E \rightarrow E+T | T$
 $T \rightarrow id.$

Right Recursion:

The grammar is said to be right recursive if the right most variable of RHS is same as the variable at LHS.

ex: 1. $S \rightarrow aS | b$

2. $S \rightarrow$

3. $E \rightarrow T+ET$
 $T \rightarrow id.$

Note:-

1. The grammar which is both left and right recursive is Ambiguous.

ex: $S \rightarrow SS | AB$
 $A \rightarrow Aa | a$
 $B \rightarrow Bb | b$

1. The Grammar $S \rightarrow SS | 0$ is

a. No ambiguous b. Ambiguous c. left recursion d. right recursion

2. $A \rightarrow AA | (A) | a$

a. No ambiguous b. Ambiguous c. left recursion d. right recursion

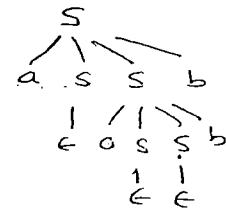
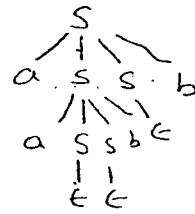
2. The recursion which neither left nor right is called General recursion.

ex: ① $A \rightarrow (A) | a$ ② $S \rightarrow aSb | bSa | e$ ③ $S \rightarrow aSSb | e$

It is of general recursion but the grammar is ambiguous

$$\text{ex: } S \rightarrow a S S b \mid \epsilon$$

$$w = a a b b$$



3. If the grammar is left recursive then the parser go to infinite loop. so to avoid the looping we need to convert the left recursive grammar into right recursive grammar.

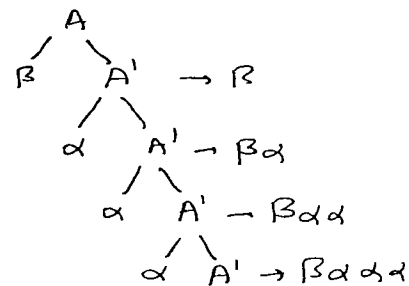
Conversion of left recursion into Right recursion.

$$LRG \rightarrow RRG.$$

$$1. A \rightarrow A\alpha \mid \beta$$

$$\Rightarrow A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$



$$\boxed{\beta \alpha^n}$$

$$2. A \rightarrow A\alpha \mid \beta_1 \mid \beta_2$$

$$\Rightarrow A \rightarrow \beta_1 A' \mid \beta_2 A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

$$3. A \rightarrow A\alpha \mid \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_n$$

$$\Rightarrow A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

$$4. A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \dots \mid A\alpha_n \mid \beta$$

$$\Rightarrow A \rightarrow \beta A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \epsilon$$

$$5. A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_m$$

$$\Rightarrow A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \epsilon$$

Remove the left recursion for the following Grammar

$$1. A \rightarrow Aa \mid b$$

$$\Rightarrow A \rightarrow bA'$$

$$A' \rightarrow aA' \mid \epsilon$$

$$2. S \rightarrow SS \mid \frac{0}{a} \mid \frac{b}{\bar{b}}$$

$$S \rightarrow OS'$$

$$S' \rightarrow SS' \mid \epsilon$$

$$3. S \rightarrow S \frac{S}{\alpha} \mid \frac{0}{\beta}$$

$$S \rightarrow OS'$$

$$S' \rightarrow SSS' \mid \epsilon$$

$$4. S \rightarrow S \frac{OSI}{\alpha} \mid \frac{\epsilon}{\beta}$$

$$S \rightarrow \epsilon S'$$

$$S' \rightarrow OSIS' \mid \epsilon$$

$$5. S \rightarrow S \frac{a S b}{\alpha_1} \mid S \frac{b S a}{\alpha_2} \mid \frac{\epsilon}{\beta}$$

$$S \rightarrow \epsilon S'$$

$$S' \rightarrow a S b S' \mid b S a S' \mid \epsilon$$

$$6. E \rightarrow E \frac{+ E}{\alpha_1} \mid E \frac{* E}{\alpha_2} \mid \epsilon$$

$$E \rightarrow id E \mid \epsilon$$

$$E' \rightarrow + E E' \mid * E E' \mid \epsilon$$

$$7. A \rightarrow \underline{A \underline{bd}} \mid \underline{A \underline{cB}} \mid \underline{d} \mid \underline{f} \mid \underline{e}$$

$\alpha_1 \quad \alpha_2 \quad \beta_1 \quad \beta_2 \quad \beta_3$

$$A \rightarrow dA' \mid fA' \mid eA'$$

$$A' \rightarrow bdA' \mid cBA' \mid \epsilon$$

$$8. E \rightarrow E + \frac{T}{2} \mid \frac{T}{3} \quad (LR)$$

$$T \rightarrow T * \frac{F}{2} \mid \frac{F}{3} \quad (LR)$$

$$F \rightarrow (E) \mid id$$

Sol: $E \rightarrow TE'$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

$$9. A \rightarrow A(A) \mid a$$

$$A \rightarrow aA'$$

$$A' \rightarrow (A)A' \mid \epsilon$$

$$10. S \rightarrow (L) \mid a$$

$$L \rightarrow L \frac{S}{2} \mid \frac{S}{3}$$

Sol

$$S \rightarrow (L) \mid a$$

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' \mid \epsilon$$

$$11. S \rightarrow AA \mid a$$

$$A \rightarrow Ab \mid SAB \mid b$$

Sol: $S \rightarrow AA \mid a$

$$A \rightarrow \underline{Ab} \mid \underline{AAAb} \mid \underline{aAb} \mid \underline{b}$$

$\alpha_1 \quad \alpha_2 \quad \beta_1 \quad \beta_2$

$$\Rightarrow S \rightarrow AA \mid a$$

$$A \rightarrow aAbA' \mid bA'$$

$$A' \rightarrow bA' \mid AAbA' \mid \epsilon$$

Non Deterministic Grammar:

The grammar with common prefixes is called non deterministic grammar.

$$A \rightarrow \alpha B_1 \mid \alpha B_2 \mid \alpha B_3$$

non deterministic

Note:

1. The grammar with common prefixes requires backtracking.
2. Backtracking is costly.
3. To avoid the backtracking we need to convert the non-deterministic grammar into deterministic we need to perform left factoring.

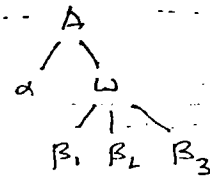
Left factoring:

The process of conversion of grammar with common prefixes into deterministic grammar is called left factoring.

Ex: $A \rightarrow \alpha B_1 \mid \alpha B_2 \mid \alpha B_3$

$\rightarrow A \rightarrow \alpha w$

$w \rightarrow B_1 \mid B_2 \mid B_3$



Ex ① $s \rightarrow \underline{a} s b \mid \underline{a} b s \mid \underline{a} b$

$s \rightarrow a s'$

$s' \rightarrow s b \mid b s \mid b$

2. $s \rightarrow a b c d \mid a b c e \mid a b f \mid a g$

(1) $s \rightarrow a b c s' \mid a b f \mid a g$

$s' \rightarrow d \mid e$

(2) $s \rightarrow a b s'' \mid a g$

$s'' \rightarrow c s' \mid f$

$s' \rightarrow d \mid e$

(3) $s \rightarrow a s'''$

$s''' \rightarrow b s'' \mid g$

$s'' \rightarrow c s' \mid f$

$s' \rightarrow d \mid e$

(4) Dangling else problem - Defining the grammar for Conditional statement.

$s \rightarrow i E t s \mid i E t s E s \mid \epsilon$

$E \rightarrow b$

Sol: $s \rightarrow i E t s s' \mid \epsilon$

$s' \rightarrow \epsilon \mid E s$

$E \rightarrow b$

Classification of parsers:

① Top down parser (TDP)

② Bottom up parser (BUP)

Top Down parser (TDP):

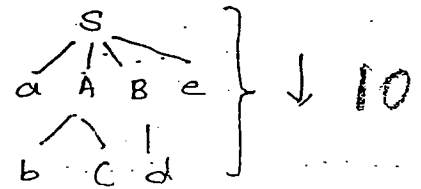
The process of construction of parse tree starting from root and proceed to children is called TDP. i.e. starting from start symbol of the grammar and reaching the i/p string.

Ex: $S \rightarrow aABc$

$A \rightarrow bc$

$B \rightarrow d$

$w = abcde$



Note:

1. TDP internally uses left most derivation.
2. TDP constructed for the grammar if it is free from ambiguity and left recursion.
3. TDP used for the grammar with less complexity if complexity is more than the parsing mechanism is slow and hence performance is low.
4. Average Time complexity is $O(n^4)$

Classification of Top down parser

① with Back Tracking

→ BruteForce Technique

② without Back Tracking

→ predictive parsers

① LL(1) or Non-recursive

② Recursive Descent parser.

BruteForce Technique:

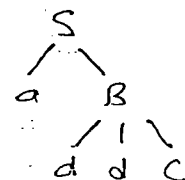
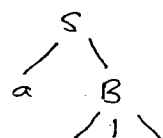
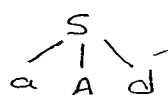
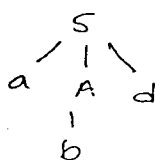
Whenever a non terminal is expanding for the first time, go with the first alternative and compare with the input string. If it does not match, go with the second alternative and compare with the input string. If it does not match, go with the third and so on, and continue with all the alternatives. If at least one alternative matches the input string, then the input string is parsed successfully. Otherwise, the string can not be parsed.

Ex: $S \rightarrow aAd | aB$

$A \rightarrow b | c$

$B \rightarrow ccd | ddc$

$w = addc$

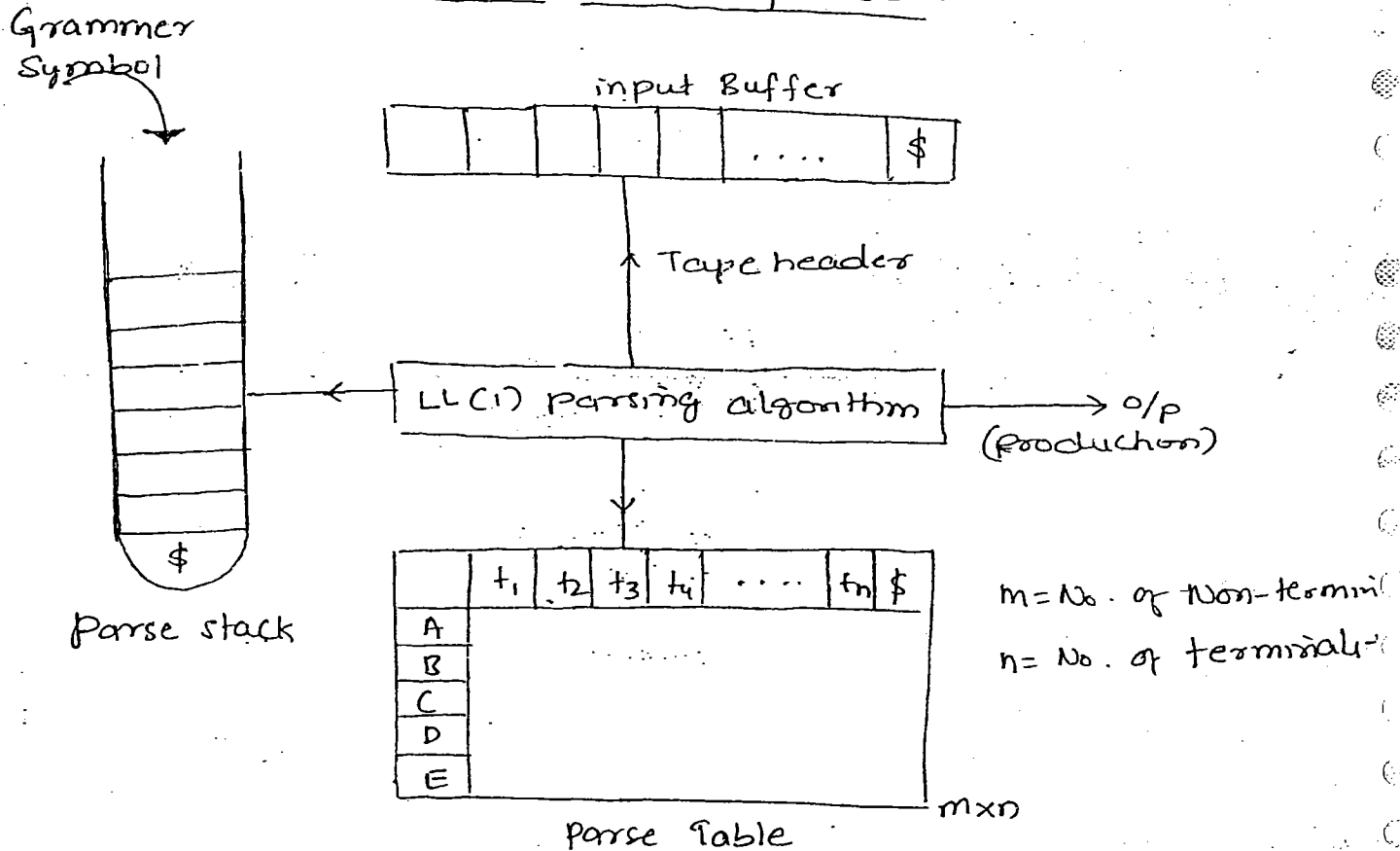


LL(1) parser (or) Table driven parser :

LL(1)

- No. of symbol taken into count for parsing
- Left to Right.
- LMD (Left most derivation)

Block Diagram of LL(1) parser :



LL(1) has

1. Input Buffer
2. Parse stack
3. Parse table

Input Buffer:

It is divided into cells and each cell is capable of holding only one symbol.

Input Buffer contains only one

The Tape header always pointing only one symbol at a time, The symbol which is pointed by tape header is called look ahead symbol.

Parse stack:

It contains the grammar symbols. The grammar symbols are pushed into stack or popped from the stack based on the occurrence of the matching. i.e. if there is no matching b/w the top most symbol of the stack and look ahead symbol then the grammar symbols are pushed into stack. if matching occurs b/w the top most symbol of the stack and look ahead symbol then the grammar symbols are popped from the stack.

Parse Table:

It is a two dimensional array of order $m \times n$ where $m = \text{No. of non-terminals}$ & $n = \text{no. of terminals} + 1$. It contains all the productions which are used in the parsing to push the grammar symbols into stack.

Parsing process:

- Step (1): Push the start symbol into the stack.
- Step (2): Compare the topmost symbol of the stack with the look ahead symbol.
- Step (3): If matching occurs then top of the grammar symbol and increment the i/p pointer & continue the process.
- Step (4): Output the production which is used for the expanding a non-terminal and continue.

LL(1) parsing Algorithm:

Let x be the grammar symbol (start symbol) in the stack and a is the look ahead symbol.

- (1) If $x = a = \$$ then, it is successful in parsing.
- (2) If $x = a \neq \$$ then Pop off and increment the i/p pointer.
- (3) If $x \neq a \neq \$$ & $m[x, a]$ has the production $x \rightarrow uvw$ then replace by uvw in the reverse order & continue the proc.
- (4) output the production which is used for expanding the non-terminal.

Ex: $S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

$w = abab$

stack	i/p string	Action
\$S	abab\$	push ($S \rightarrow AA$)
\$AA	↑ abab\$	push ($A \rightarrow aA$)
\$AAa	↑ abab\$	pop
\$AA	bab\$	push ($A \rightarrow b$)
\$Ab	bab\$	pop
\$A	ab\$	push ($A \rightarrow aA$)
\$Aa	ab\$	pop
\$A	b\$	push ($A \rightarrow b$)
\$b	b\$	pop
\$	·\$	Accept

	a	b	\$
S	$S \rightarrow AA$	$S \rightarrow AA$	
A	$A \rightarrow aA$	$A \rightarrow b$	

LL(1) Grammar:

The grammar G is said to be for which LL(1) parser can be constructed is known as LL(1) grammar.

(or)

The grammar whose parse table does not contain the multiple entries is called LL(1) grammar.

Functions used for the construction of parse table

① First(α) → Terminal/NonTerminal.

② Follow(A). Non Terminal.

FIRST(α):

First(α) is the set of all terminals that may begin in any sentential form which is derived from α

12

Rules:

1. If α is a terminal then

$$\text{First}(\alpha) = \{\alpha\}$$

2. If α is a Non terminal and defined by the rule $\alpha \rightarrow \epsilon$ then

$$\text{First}(\alpha) = \{\epsilon\}$$

3. If α is a Non terminal and defined by non null pro i.e

$$\alpha \rightarrow x_1 x_2 x_3$$

$$\text{First}(\alpha) = \text{First}(x_1)$$

$$= \text{First}(x_1) \cup \text{First}(x_2)$$

$$= \text{First}(x_1) \cup \text{First}(x_2) \cup \text{First}(x_3)$$

$$= \text{First}(x_1) \cup \text{First}(x_2) \cup \text{First}(x_3) \cup \{\epsilon\}$$

Ex:

① $S \rightarrow \epsilon$

$$\text{First}(S) = \{\epsilon\}$$

② $S \rightarrow a | \epsilon$

$$\text{First}(S) = \{a, \epsilon\}$$

③ $S \rightarrow aA | \epsilon$ $F(S) = \{a, \epsilon\}$
 $A \rightarrow b$ $F(A) = \{b\}$

④ $S \rightarrow aAB$ $F(S) = \{a\}$

$$A \rightarrow bB | c$$
 $F(A) = \{b, c\}$

$$B \rightarrow dA | a$$
 $F(B) = \{d, a\}$

⑤ $S \rightarrow aAB$ $F(S) = \{a\}$
 $A \rightarrow b | \epsilon$ $F(A) = \{b, \epsilon\}$
 $B \rightarrow a$ $F(B) = \{a\}$

⑥ $S \rightarrow AB$ $F(S) = \{a, b, \epsilon\}$

$$A \rightarrow a | \epsilon$$
 $F(A) = \{a, \epsilon\}$

$$B \rightarrow b | \epsilon$$
 $F(B) = \{b, \epsilon\}$

⑦ $S \rightarrow ABCDE$

$$A \rightarrow a | \epsilon$$

$$B \rightarrow b | \epsilon$$

$$C \rightarrow c | \epsilon$$

$$D \rightarrow d$$

$$E \rightarrow e | \epsilon$$

	First
S	{a, b, c, d, e}
A	{a, \epsilon}
B	{b, \epsilon}
C	{c, \epsilon}
D	{d}
E	{e, \epsilon}

⑧ $S \rightarrow ABCDE$

$$A \rightarrow a | \epsilon$$

$$B \rightarrow b | \epsilon$$

$$C \rightarrow c | \epsilon$$

$$D \rightarrow d | \epsilon$$

$$E \rightarrow e | \epsilon$$

	First
S	{a, b, c, d, e, \epsilon}
A	{a, \epsilon}
B	{b, \epsilon}
C	{c, \epsilon}
D	{d, \epsilon}
E	{e, \epsilon}

⑨

$$S \rightarrow aABd$$

$$F(S) = \{a\}$$

$$A \rightarrow BA | \epsilon$$

$$F(A) = \{\epsilon, d, b\}$$

FOLLOW(A):

$\text{Follow}(A)$ is the set of all the terminals that may follow to the right of A in any sentential form of the grammar.

13

Rules to find $\text{Follow}(A)$:

1. If A is a start symbol, then

$$\text{Follow}(A) = \{\$ \}$$

2. If $S \rightarrow \alpha A \beta$ ($\beta \neq \epsilon$)

grammar
symbols

$$\text{follow}(A) = \text{First}(\beta) \text{ (without } \epsilon \text{)}$$

3. If $S \rightarrow \alpha A \alpha$ or

$$S \rightarrow \alpha A \beta \quad (\beta \neq \epsilon)$$

then, $\text{follow}(A) = \text{Follow}(S)$.

ex: ① $S \rightarrow aA$

$$A \rightarrow b$$

$$\text{Follow}(S) = \{\$ \}$$

$$\text{Follow}(A) = \{\$ \}$$

② $S \rightarrow aAb$

$$A \rightarrow Ba|b$$

$$B \rightarrow d$$

$$\text{Follow}(S) = \{\$ \}$$

$$\text{Follow}(A) = \{b\}$$

$$\text{Follow}(B) = \{a\}$$

③ $S \rightarrow SOSI|E$

$$\text{Follow}(S) = \{\$, 0, 1\}$$

④ $S \rightarrow AaAb|BaBb$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

$$\text{Follow}(S) = \{\$ \}$$

$$\text{Follow}(A) = \{a, b\}$$

$$\text{Follow}(B) = \{a, b\}$$

⑤ $S \rightarrow AB$

$$A \rightarrow a|\epsilon$$

$$B \rightarrow b|\epsilon$$

$$\text{follow}(S) = \{\$ \}$$

$$\text{follow}(A) = \{b, \$ \}$$

$$\text{follow}(B) = \{\$ \}$$

⑥ $S \rightarrow ABCDE$

$$A \rightarrow a|\epsilon$$

$$B \rightarrow b|\epsilon$$

$$C \rightarrow c|\epsilon$$

$$D \rightarrow d|\epsilon$$

$$E \rightarrow \epsilon$$

	Follow
S	$\{\$ \}$
A	$\{b, c, d, e\}$
B	$\{c, d, e\}$
C	$\{d, e\}$
D	$\{e\}$
E	$\{\$ \}$

$$8. \quad S \rightarrow aBDh$$

$$B \rightarrow cC$$

$$C \rightarrow bC \mid \epsilon$$

$$D \rightarrow EF$$

$$E \rightarrow g \mid \epsilon$$

$$F \rightarrow f \mid \epsilon$$

$$\text{Follow}(S) = \{ \$ \}$$

$$\text{Follow}(B) = \{ g, f, h \}$$

$$\text{Follow}(C) = \{ g, f, h \}$$

$$\text{Follow}(D) = \{ h \}$$

$$\text{Follow}(E) = \{ f, h \}$$

$$\text{Follow}(F) = \{ h \}$$

$$9. \quad A \rightarrow (A).B \mid a$$

$$B \rightarrow aBa \mid b$$

$$\text{Follow}(A) = \{ \$,) \}$$

$$\text{Follow}(B) = \{ a, \$,) \}$$

$$10. \quad S \rightarrow aBCbD$$

$$B \rightarrow bBh \mid BD \mid d$$

$$C \rightarrow CE \mid a$$

$$D \rightarrow EC \mid b \mid \epsilon$$

$$E \rightarrow CDB \mid \epsilon$$

$$\text{Follow}(S) = \{ \$ \}$$

$$\text{Follow}(B) = \{ a, h, b \}$$

$$\text{Follow}(C) = \{ b, a \}$$

$$\text{Follow}(D) = \{ \$, a, h, b \}$$

$$\text{Follow}(E) = \{ b, a \}$$

construct the following set for ϵ and every variables for the following grammar after elimination of left recursion.

$$1. \quad \left. \begin{array}{l} E \rightarrow E + T \mid T \\ T \rightarrow id \end{array} \right\} \text{ not LL(1)}$$

Sol:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow id$$

$$\text{Follow}(E) = \{ \$ \}$$

$$\text{Follow}(E') = \{ \$ \}$$

$$\text{Follow}(T) = \{ +, \$ \}$$

$$2. \quad E \rightarrow E + E \mid E * E \mid id$$

$$E \rightarrow idE'$$

$$E' \rightarrow +EE' \mid *EE' \mid \epsilon$$

$$\text{Follow}(E) = \{ \$, +, * \}$$

$$\text{Follow}(E') = \{ \$, +, * \}$$

$$3. \quad E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

$$\text{Sol: } E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

$$\text{Follow}(E) = \{ \$,) \}$$

$$\text{Follow}(E') = \{ \$,) \}$$

$$\text{Follow}(T) = \{ +, \$,) \}$$

$$\text{Follow}(T') = \{ +, \$,) \}$$

$$\text{Follow}(F) = \{ *, +, \$,) \}$$

$$4. S \rightarrow AA$$

$$A \rightarrow aA$$

$$A \rightarrow b$$

$$\text{fol}(S) = \{\$ \}$$

$$\text{fol}(A) = \{a, b, \$ \}$$

$$5. S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

$$\text{sol: } S \rightarrow (L) | a$$

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' | \epsilon$$

$$\text{fol}(L) = \{ \$,), , \}$$

$$\text{fol}(L) = \{ \$,) \}$$

$$\text{fol}(L') = \{ \$,) \}$$

$$6. R \rightarrow R + R | RR | R * (CR) | \underline{a} | \underline{b}$$

β, β, β_3

$$\text{sol: } R \rightarrow (R) R' | a R' | b R'$$

$$R' \rightarrow + R R' | R R' | * R' | \epsilon$$

$$\text{fol}(R) = \{ \$,), +, *, (, a, b \}$$

$$\text{fol}(R') = \{ \text{fol}(R) \}$$

$$7. S \rightarrow i E + S S' | a$$

$$S' \rightarrow e S | \epsilon$$

$$E \rightarrow b$$

$$\text{fol}(S) = \{ \$, e \}$$

$$\text{fol}(S') = \{ \$, e \}$$

$$\text{fol}(\epsilon) = \{ \$ \}$$

Procedure to construct the LL(1) parser.

- For every production $A \rightarrow \alpha$ repeat the following steps
1. Add $A \rightarrow \alpha$ in $m[A, a]$ for every symbol a in $\text{First}(\alpha)$
 2. $\text{First}(\alpha)$ contains ϵ and Add $A \rightarrow \alpha$ in $m[A, b]$ for every symbol b in $\text{FOLLOW}(A)$.

Note:

The grammar, 'G' is LL(1) iff its parse table does not contains multiple entries.

1. ex:

	First	
$S \rightarrow iEtSS' \mid a$		$\text{fol}(S) = \{ \$, c \}$
$S' \rightarrow es \mid \epsilon$	S $\{ i, a \}$	$\text{fol}(S') = \{ \$, e \}$
$E \rightarrow b$	S' $\{ e, \epsilon \}$	$\text{fol}(E) = \{ t \}$
	E $\{ b \}$	

$$V = \{ i, t, a, c, b, \$ \}$$

	i	a	b	e	t	\$
S	$S \rightarrow iEtSS'$	$S \rightarrow a$				
S'				$S' \rightarrow es$ $S' \rightarrow \epsilon$		$S' \rightarrow \epsilon$
E			$E \rightarrow b$			

$\therefore$ Grammar is not LL(1).

2. $R \rightarrow R+R \mid RR \mid R* \mid (R) \mid a \mid b$ (Not LL(1))

$$R \rightarrow (R)R' \mid aR' \mid bR'$$

$$R' \rightarrow +RR' \mid RR' \mid *R' \mid \epsilon$$

$$F(R) = \{ (, a, b \}$$

$$\text{fol}(R) = \{ \$,), +, *, (\}$$

$$F(R') = \{ *, +, \epsilon, (, a, b \}$$

$$\text{fol}(R') = \{ \text{fol}(R) \}$$

	(	)	+	*	a	b	\$
R	$R \rightarrow (R)R'$				$R \rightarrow aR'$	$R \rightarrow bR'$	
R'	$R' \rightarrow +RR'$ $R' \rightarrow \epsilon$	$R' \rightarrow \epsilon$	$R' \rightarrow +RR'$ $R' \rightarrow \epsilon$	$R' \rightarrow *RR'$ $R' \rightarrow \epsilon$	$R' \rightarrow \epsilon$ $R' \rightarrow RR'$	$R' \rightarrow \epsilon$ $R' \rightarrow RR'$	$R' \rightarrow \epsilon$

$$3. \quad S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

$$F(S) = \{ (, a \}$$

$$fol(S) = \{ \$,), a \}$$

$$F(L) = \{ (, a \}$$

$$fol(L) = \{ \$,) \}$$

$$F(L') = \{ ,, \epsilon \}$$

$$fol(L') = \{ \$,) \}$$

Sol: $S \rightarrow (L) | a$

$$L \rightarrow SL'$$

$$L' \rightarrow ,, SL' | \epsilon$$

	(	)	a	,	\$
S	$S \rightarrow (L)$		$S \rightarrow a$		
L	$L \rightarrow SL'$		$L \rightarrow SL'$		
L'		$L' \rightarrow \epsilon$		$L' \rightarrow ,, SL'$	$L' \rightarrow \epsilon$

$\therefore LL(1)$

$$4. \quad S \rightarrow AA$$

$$A \rightarrow aA$$

$$A \rightarrow b$$

$$F(S) = \{ a, b \}$$

$$fol(S) = \{ \$ \}$$

$$F(A) = \{ a, b \}$$

$$fol(A) = \{ a, b, \$ \}$$

	a	b	\$
S	$S \rightarrow AA$	$S \rightarrow AA$	
A	$A \rightarrow aA$	$A \rightarrow b$	

$\therefore LL(1)$

$$5. \quad E \rightarrow E + T | T$$

$$T \rightarrow int$$

$$F(E) = \{ int \}$$

$$fol(E) = \{ \$ \}$$

$$F(E') = \{ +, \epsilon \}$$

$$fol(E') = \{ \$ \}$$

$$F(T) = \{ int \}$$

$$fol(T) = \{ +, \$ \}$$

Sol: $E \rightarrow TE'$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow int$$

	+	int	\$
E		$E \rightarrow TE'$	
E'	$E' \rightarrow +TE'$		$E' \rightarrow \epsilon$
T		$T \rightarrow int$	

6. $E \rightarrow TE'$

	First	Follow
$E' \rightarrow +TE' \epsilon$	$E: \{ (, id \}$	$\{ \$,) \}$
$T \rightarrow FT'$	$E': \{ +, \epsilon \}$	$\{ \$,) \}$
$T' \rightarrow *FT' \epsilon$	$T: \{ (, id \}$	$\{ +, \$,) \}$
$F \rightarrow (E) id$	$T': \{ *, \epsilon \}$	$\{ +, \$,) \}$
	$F: \{ (, id \}$	$\{ *, +, \$,) \}$

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Note:

Shortcut to find LL(1) parser

1. A grammar without ϵ rule is LL(1) for each production of the form $A \rightarrow \alpha_1 | \alpha_2 | \alpha_3 | \dots | \alpha_n$

$$\text{First}(\alpha_1) \cap \text{First}(\alpha_2) \cap \dots \cap \text{First}(\alpha_n) = \phi$$

i.e. $F(\alpha_1), F(\alpha_2), \dots, F(\alpha_n)$ are mutually disjoint.

$$\text{First}(\alpha_i) \cap \text{First}(\alpha_j) = \phi$$

2. The grammar with ϵ rule is LL(1) for each rule of the form $A \rightarrow \alpha | \epsilon$

$$\text{First}(\alpha) \cap \text{Follow}(A) = \phi$$

3. Ambiguous grammar is not LL(1)
4. Left recursive grammar is not LL(1)
5. Non left factor grammar is not LL(1)

6. The grammar in which every production has only one alternative RHS is always LL(1)

ex:

$S \rightarrow AB$

$A \rightarrow aB$

$B \rightarrow b$

	a	b	S
S	$S \rightarrow AB$	$S \rightarrow AB$	
A	$A \rightarrow aB$		
B		$B \rightarrow b$	

$A \rightarrow c$

$B \rightarrow d$

$2 \rightarrow$

2. $S \rightarrow aA|bB$

$$\{a\} \cap \{b\} = \emptyset$$

$A \rightarrow Bb|a$

$$\{b, c\} \cap \{a\} = \emptyset$$

$B \rightarrow bB|c$

$$\{b\} \cap \{c\} = \emptyset$$

$\Rightarrow$ So grammar is LL(1)

3. $S \rightarrow AB|BA$

$$\{a, b\} \cap \{a, b\} \neq \emptyset$$

$A \rightarrow a|b$

$B \rightarrow b|a$

$\Rightarrow$ So grammar is not LL(1)

4. $S \rightarrow aABb$

$$\{a\} \cap \{b\} = \emptyset$$

$A \rightarrow a|e$

$$\{b\} \cap \{e\} = \emptyset$$

$B \rightarrow b|e$

$\Rightarrow$ So grammar is LL(1)

5. $S \rightarrow AaAb|BbBa$

$A \rightarrow e$

$$\{a\} \cap \{b\} = \emptyset$$

$B \rightarrow e$

$\Rightarrow$ So grammar is LL(1)

6. $S \rightarrow AaAb|BaBb$

$A \rightarrow e$

$$\{a\} \cap \{a\} \neq \emptyset$$

$B \rightarrow e$

Verify the following grammars is LL(1) or Not.

1. $S \rightarrow aABb$

$A \rightarrow c|e$

$$\{c\} \cap \{d, b\} = \emptyset$$

$B \rightarrow d|e$

$$\{d\} \cap \{b\} = \emptyset$$

$$2. S \rightarrow ACB | CbB | Ba$$

$$A \rightarrow da | BC$$

$$B \rightarrow g | \epsilon$$

$$C \rightarrow h | \epsilon$$

$$\{d, g, h\} \cap \{b, \dots\} \neq \emptyset$$

$\Rightarrow$ So grammar is not LL(1).

$$3. S \rightarrow \underline{I} AB | \epsilon$$

$$A \rightarrow \underline{I} AC | DC$$

$$B \rightarrow OS$$

$$C \rightarrow I$$

$$I \cap \epsilon = \emptyset$$

LL(1)

GATE:

$$1. S \rightarrow A | a$$

$$A \rightarrow a$$

$\Rightarrow$ Not LL(1)

$$2. \text{pgm} \rightarrow \text{begin } d \text{ semi } x \text{ end}$$

$$x \rightarrow d \text{ semi } x | s \ y \quad \{d\} \cap \{s\} = \emptyset$$

$$y \rightarrow \text{semi } s \ y | \epsilon \quad \{\text{semi}\} \cap \{\text{end}\} = \emptyset$$

$\Rightarrow$ LL(1)

3. find the first and follow

04

$$E \rightarrow aA | (E)$$

$$A \rightarrow +E | *E | \epsilon$$

	first
E	{a, (}
A	{+, *, } }

	Follow
E	{ \$,) }
A	{ \$,) }

$$4. S \rightarrow iE + SS' | \epsilon$$

$$05 \quad S' \rightarrow cS | \epsilon$$

$$E \rightarrow b$$

$$\{i\} \cap \{a\} = \emptyset$$

$$\{c\} \cap \{e\} \neq \emptyset$$

$\Rightarrow$ Not LL(1)

$$5. A \rightarrow AA | (A) | \epsilon$$

06

(a) Left recursion (b) Right recursion (c) Ambiguous (d) None

$$06. S \rightarrow FR$$

$$R \rightarrow *S | \epsilon$$

$$F \rightarrow id$$

Parcing table Entries

$m[s, id]$ and $m[R, F]$

$m[s, id]$

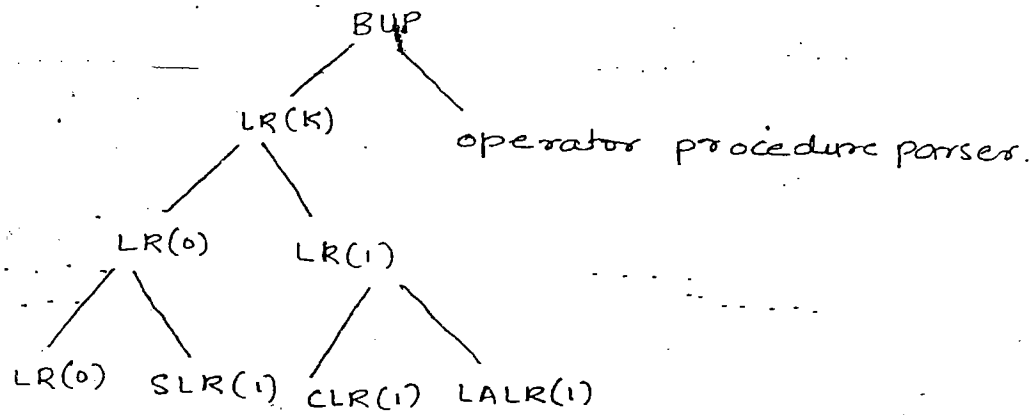
$\downarrow$
 $S \rightarrow FR$

$m[R, F]$

$\downarrow$
 $R \rightarrow \epsilon$

*** BOTTOM UP PARSER

17

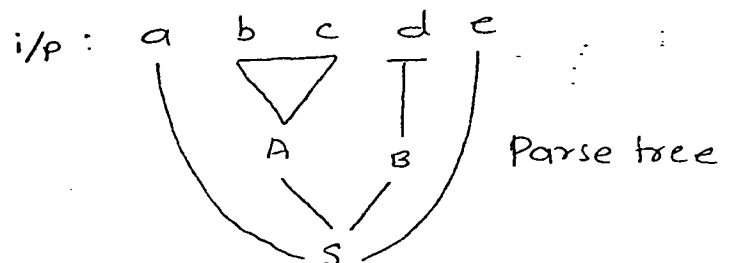


Unambiguous	operator procedure parser
LR(0)	- only operator grammar
SLR(1)	- both ambiguous and unambiguous
CLR(1) LR(1)	
LALR(1)	

Bottom up parser (BUP):

The process of construction of the parse tree in the bottom up manner i.e. starting with the children and proceeds to the root is called Bottom up parser.

ex: $S \rightarrow aABe$
 $A \rightarrow bc|b$
 $B \rightarrow d$



Handle:

substring of the given i/p string that matches with R_i of any production is called Handle.

Note:

1. BUP is also called a shift reduce parser.
2. The BUP uses reverse of rightmost derivation.
3. It has to detect the handle and reduce the handle.
4. Detecting the handle is main overhead.
5. BUP is constructed for Unambiguous grammar.
(except for operator grammar)

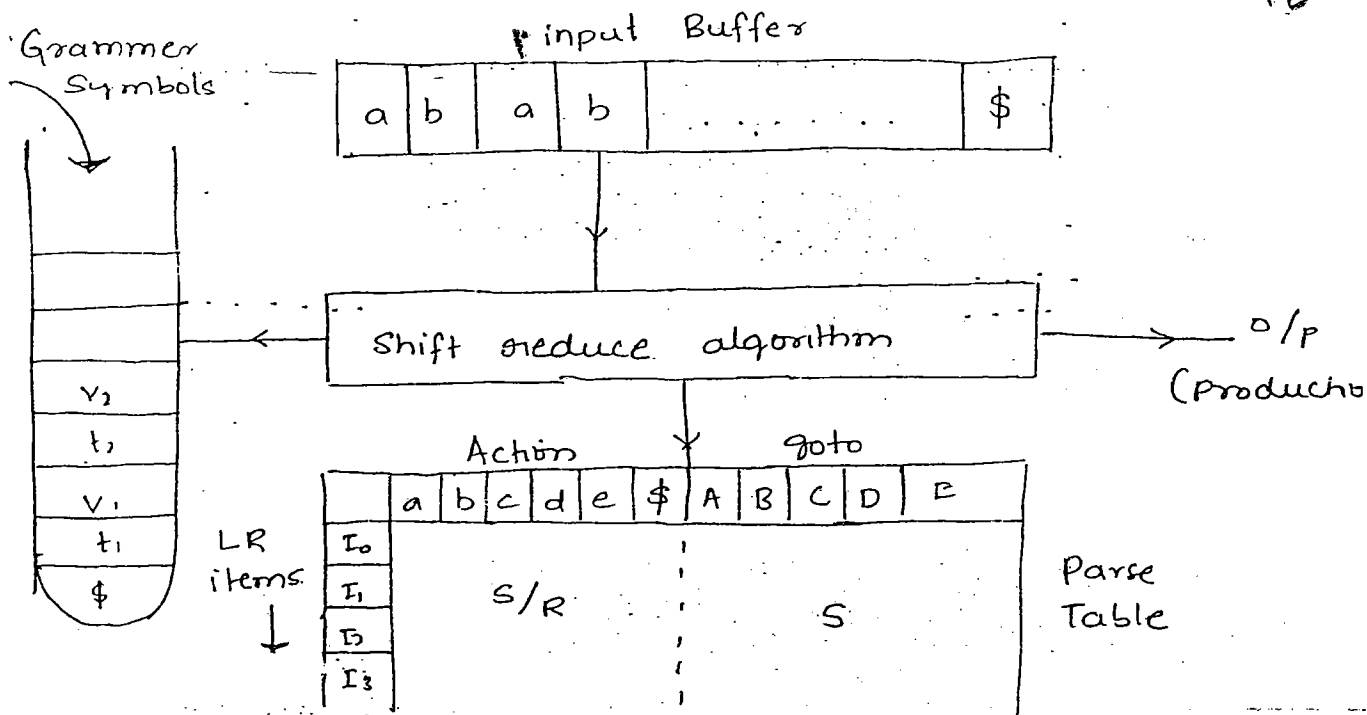
6. The BUP can be constructed for the grammar which has more complexity.
7. This Parsing mechanism is faster than Top down parsing i.e. BUP is more efficient than TDP.
8. BUP performance is high.
9. Average time complexity is $O(n^3)$.

ex: $S \rightarrow AA$
 $A \rightarrow aA$
 $A \rightarrow b$

stack	i/p	Action
\$	abab\$	shift
\$a	bab\$	shift
\$ab	ab\$	reduce $A \rightarrow b$
\$aA	ab\$	reduce $A \rightarrow aA$
\$A	ab\$	shift
\$Aa	b\$	reduce $A \rightarrow b$ shift
\$Aab	\$	reduce $A \rightarrow bA$
\$AaA	\$	reduce $A \rightarrow aA$
\$AA	\$	reduce $S \rightarrow AA$
\$S	\$	accept.

Block diagram of BUP:

18



1. Input Buffer

2. parse stack.

3. parse table

Input Buffer: consists of the i/p string to be parsed and the i/p string is ends with \$.

Parse stack: stack consist of the grammar symbols. The grammar symbols are inserted or removed from the stack using shift and reduce operations.

Parse table: parse table is constructed with no. of terminals, no. of non terminals & LR items / compiler items.

The Parse table consist of two parts:

- Action
- goto.

Action part consist of shift and reduce operations implied for the terminals. and goto consist of the shift operation which are applied on the non-terminals.

operations

- ① Shift
- ② Reduce
- ③ Accept
- ④ Error

Shift:

Shift operation can be applied whenever handle doesn't occur from the top symbol of the stack.

Reduce:

Reduce operation can be used whenever handle occurs from the top symbols of the stack.

Accept:

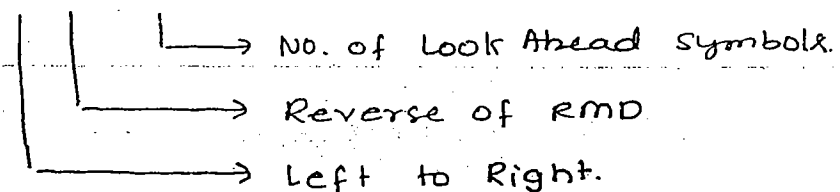
After processing the complete i/p string if the stack contains only start from the grammar then the i/p string is accepted and parse is successfully.

Error:

After processing the i/p string if stack does not contain start symbol of the grammar then the i/p string is not parse. i.e. parse tree is not constructed. So the result is error.

LR Parser

LR(K)



Classification of LR Parser:

Based on the construction of the table LR parsers are divided into 4 types

1. LR(0)
2. SLR(1)
3. CLR(1)
4. LALR(1)

Procedure for the construction of LR parse table.

19

1. Obtain the augmented grammar for the given grammar
2. Create the canonical collection of LR items or compiler
3. Grammar DFA and prepare the table based on the LR item

Note:

LR of the parser allow all the ambiguous grammars in their parsing the grammar should be taken as it is and should not be modified.

Augmented Grammar:

The grammar which is obtained by adding one more production that derives the start symbol of the grammar is called Augmented Grammar.

ex:

$S' \rightarrow S$

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

} Grammar } Augmented Grammar

Augmented grammar helps us in separating the final item from non-final item the need for augmented grammar if you have multiple production for the start symbol then we can decide what is the final string.

LR(0) items (or) items in compiler:

A production '.' at any point in the right hand side is called as LR(0) item.

ex:

$A \rightarrow abc$

$A \rightarrow .abc$

$A \rightarrow a.bc$

$A \rightarrow ab.c$

$A \rightarrow abc.$ Final / complete item

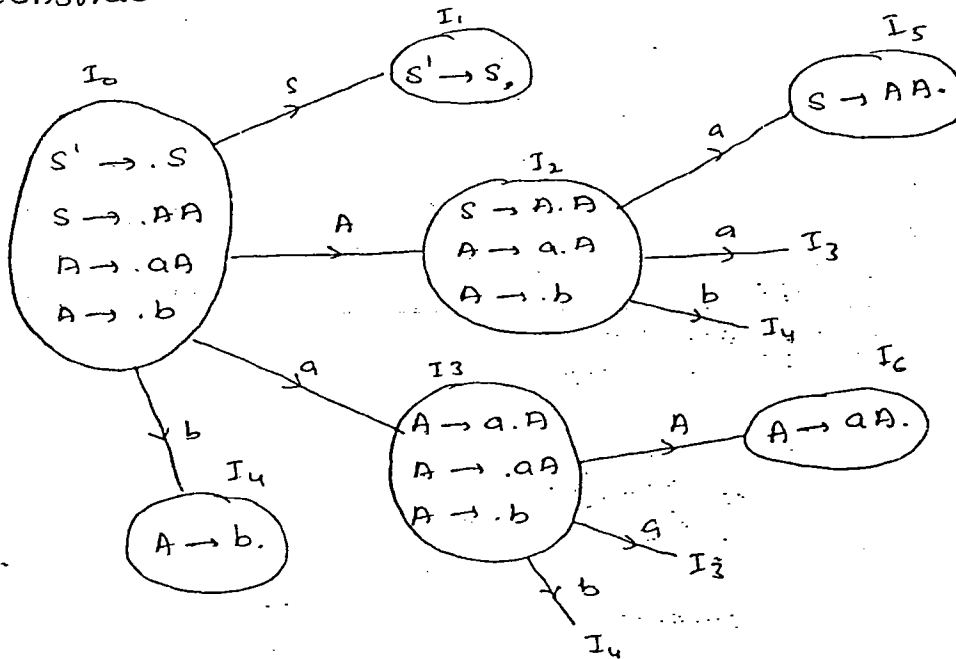
Canonical collection:

$C = \{$

$$C = \{I_0, I_1, I_2, I_3, I_4, I_5\}$$

I_0	I_1	I_2	I_3	I_4	I_5	I_6
$S' \rightarrow .S$	$S' \rightarrow .S$	$S \rightarrow A.A$	$A \rightarrow a.A$	$A \rightarrow b.$	$S \rightarrow AA.$	$A \rightarrow aA.$
$S \rightarrow .AA$		$A \rightarrow a.A$	$A \rightarrow a.A$			
$A \rightarrow .aA$		$A \rightarrow .b$	$A \rightarrow .b$			
$A \rightarrow .b$						

Construction of DFA.



Construction of LR(0) parse Table:

Parse Table consist of 2 parts:

- (1) Action
- (2) Goto

	Terminal Action	Non-terminal Goto

① $\text{goto}(I_i, x) = I_j$ x - Terminal.

	x
I_i	$S_j(I_j)$

2. Goto (I_i, x) = I_j x - Non Terminal:-

	x
I_i	I_j

3. If I_i is any final item's step then the rule g_i

	Action					
	t_1	t_2		t_n		$\$$
I_i	g_i	g_i		g_i	g_i	

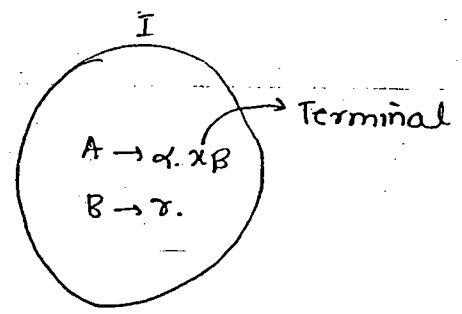
LR(0)

The grammar G is said to be LR(0) if it is parseable & free from multiple entries or conflicts.

Conflicts in LR(0):

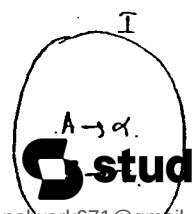
① SR-conflict:

If any state in DFA contains both shift & reduce option then it is SR-conflict.



2. RR-Conflict:

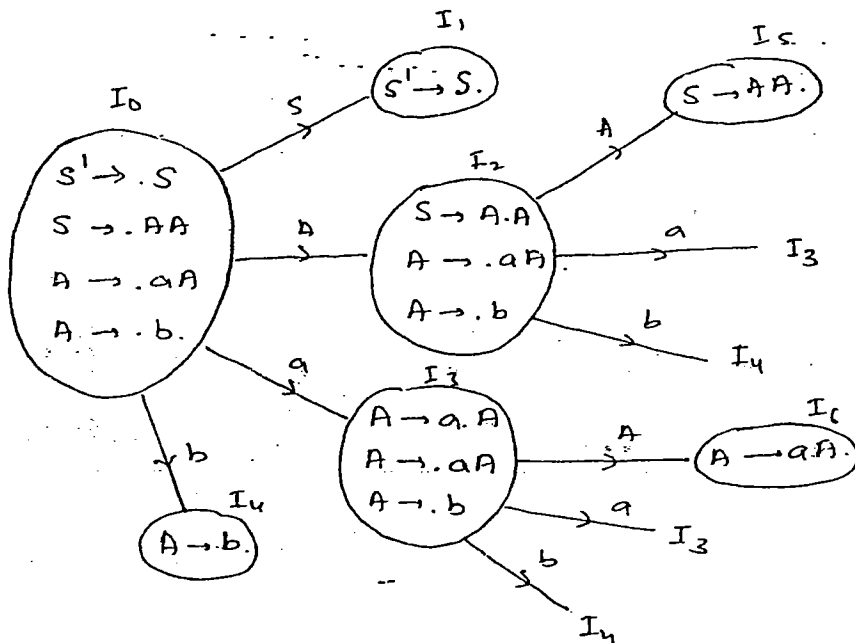
If the same state contains more than one final item then it is RR conflict.



ex: ① $S \rightarrow AB \cdot \eta_1$

$A \rightarrow aA \cdot \eta_2$

$A \rightarrow b \cdot \eta_3$

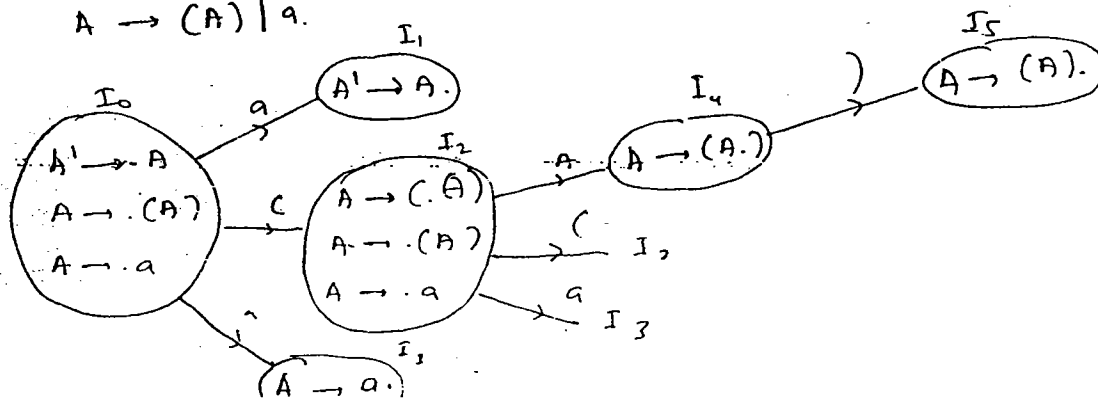


Parse Table:

	Action			Goto	
	a	b	ϕ	S	A
I_0	S_3	S_4		1	2
I_1	Acc	Acc	Acc		
I_2	S_2	S_4			5
I_3	S_2	S_4			6
I_4	η_3	η_3	η_3		
I_5	η_1	η_1	η_1		
I_6	η_2	η_2	η_2		

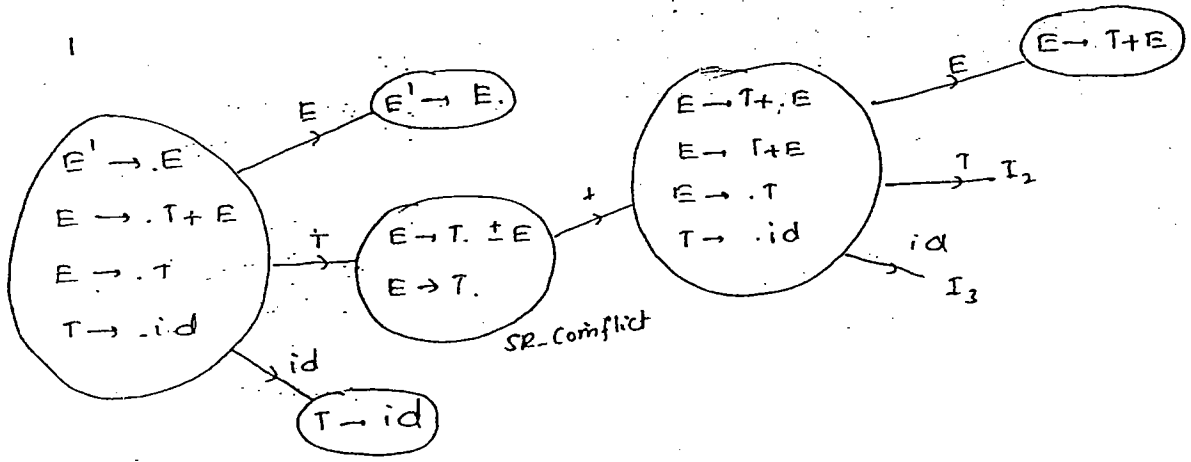
$\Rightarrow$ It is LR(0) Grammar

2. $A \rightarrow (A) \mid a$



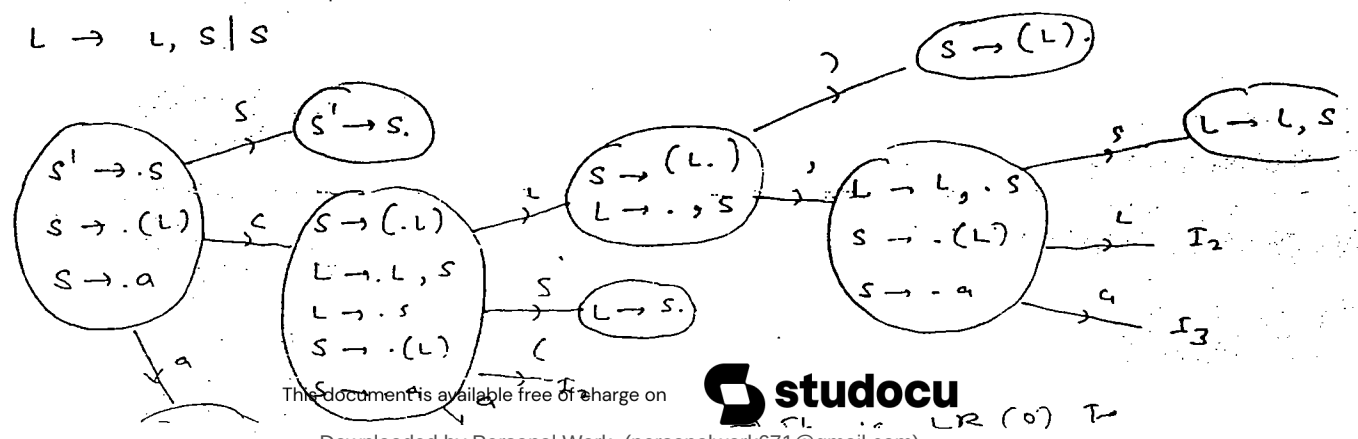
	(	)	a	\$	A
I ₀	S ₂		S ₃		1
I ₁	Acc	Acc	Acc	Acc	
I ₂	S ₂		S ₃		4
I ₃	g ₂	g ₂	g ₂	g ₂	
I ₄		S ₅			
I ₅	g ₁	g ₁	g ₁	g ₁	

3. $E \rightarrow T + E / T$
 $T \rightarrow id$



	+	id	\$	E	T
I ₀		S ₂		1	2
I ₁	Acc	Acc	Acc		
I ₂	S ₄ (r ₂)	g ₂	g ₂		
I ₃	g ₃	g ₃	g ₃		
I ₄		S ₃		5	2
I ₅	g ₁	g ₁	g ₁		

4. $S \rightarrow (L) | a$
 $L \rightarrow L, S | S$



Verify the following grammars LR(0) or not.

1. $S \rightarrow AaAb \mid BbBa$
 $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$
2. $S \rightarrow aAB \mid Ba \mid Ab$
 $A \rightarrow c$
 $B \rightarrow c$
3. $S \rightarrow Aab \mid \epsilon$
 $A \rightarrow aA \mid a$
 $B \rightarrow Ba \mid b$

4. $E \rightarrow E+T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow Id$
5. $E \rightarrow E+T \mid T$
 $T \rightarrow EF \mid Ea \mid b$
 $F \rightarrow (E) \mid a$
6. $S \rightarrow AB \mid BA$
 $A \rightarrow Aab \mid b$
 $B \rightarrow BaA \mid a$

7. $S \rightarrow A\#a \mid @bB$
 $A \rightarrow \#A \mid \#$
 $B \rightarrow *B\# \mid \#$
8. $S \rightarrow Aab \mid bab \mid bac \mid acb$
 $A \rightarrow aBA \mid b$
 $B \rightarrow b$
9. $E \rightarrow EF \mid e$
 $F \rightarrow fT \mid f$
 $T \rightarrow ET \mid g$

10. $A \rightarrow A(A) \mid bA \mid a$

1. $S' \rightarrow S$
 $S \rightarrow AaAb$
 $S \rightarrow BbBa$
 $A \rightarrow \epsilon$
 $B \rightarrow \epsilon$
 $\rightarrow$ Not LR(0)
more than one final state

2. $S' \rightarrow S$
 $S \rightarrow aAB$
 $S \rightarrow Ba$
 $S \rightarrow Ab$
 $A \rightarrow c$
 $B \rightarrow c$
RR conflict

3. $S' \rightarrow S$
 $S \rightarrow Aab$
 $S \rightarrow Ba$
 $A \rightarrow aA$
 $A \rightarrow a$
 $S' \rightarrow S$
 $S \rightarrow A.ab$
 $S \rightarrow B.a$
 $A \rightarrow a.$
 $A \rightarrow$

Note:

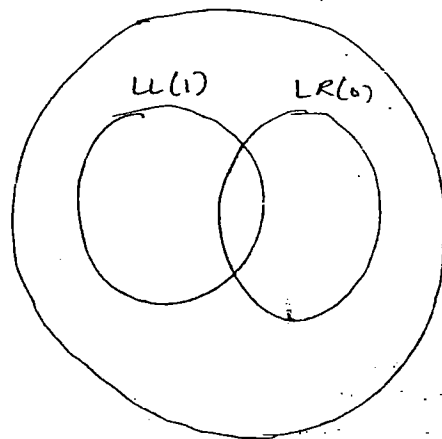
$LR(0)$ ^{grammar} need not be $LL(1)$

$LL(1)$ grammar need not be $LR(0)$

Some grammar are both $LR(0)$ and $LL(1)$.

For some grammar we can not construct $LR(0)$ & $LL(1)$.

UnAmbiguous



SLR(1) parser

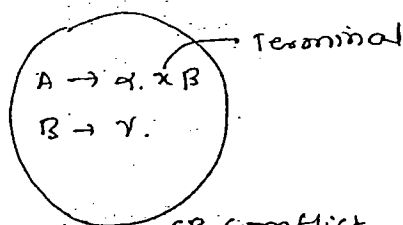
The procedure for constructing the parse table is similar to $LR(0)$ parser, but there is one restriction on reducing the entries.

Whenever there is a final item then place the reduce entries under the follow symbols of left hand side variable.

If the $SLR(1)$ parse table does not contain multiple entries that is free from conflict is SLR(1) Grammar.

Conflict in $SLR(1)$:

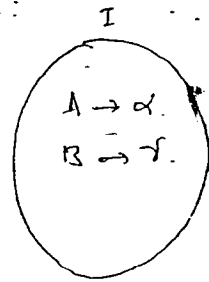
1. SR Conflict:



if $fol(B) \cap \{x\} \neq \emptyset$
then it is SR conflict
in $SLR(1)$

if $fol(B) \cap \{x\} = \emptyset$
then it is SR conflict
in $LR(0)$ and not $SLR(1)$

3. RR Conflict:



RR Conflict
on LR(0).

$$\text{If } \text{fol}(A) \cap \text{fol}(B) \neq \emptyset$$

$\Rightarrow$ It is RR conflict in SLR(1).

$$\text{If } \text{fol}(A) \cap \text{fol}(B) = \emptyset$$

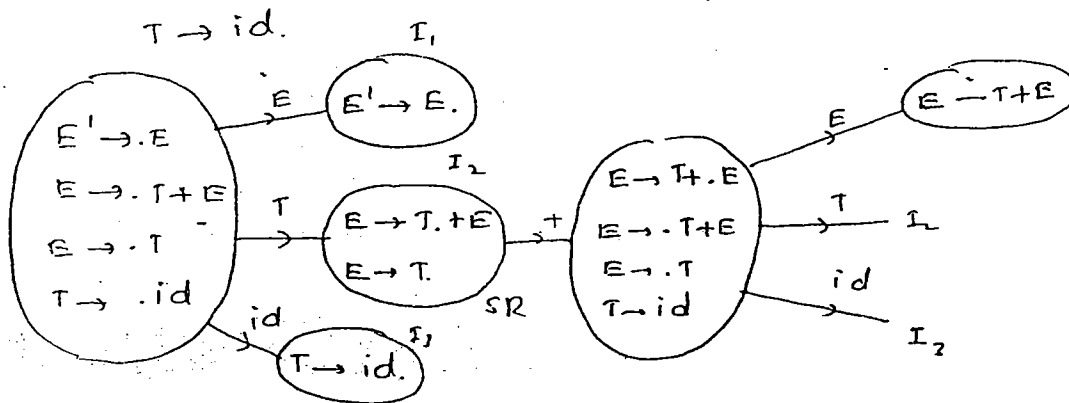
$\Rightarrow$ Then it is RR-conflict in LR but not in SLR(1).

Notes

1. Every LR(0) grammar is SLR(1).
2. Every SLR(1) grammar need not be LR(0).
3. The no. of entries in SLR(1) parse table is $\leq$ no. of ent in LR(0) parse table
4. SLR(1) parser is more powerful than LR(0).

Ex: ① $E \rightarrow T + E \mid T$.

$$\text{fol}(E) = \{\$, \cdot\} \cap \{+, \cdot\} = \emptyset$$



not LR(0), SLR(1)

	+	id	\$	E	T
I ₀		S ₃		1	2
I ₁	Acc	Acc	Acc		
I ₂	S ₄				
I ₃	S ₃		S ₃		
I ₄		S ₃		S	2
I ₅			S ₁		

2. $S \rightarrow AaAb | BbBa$

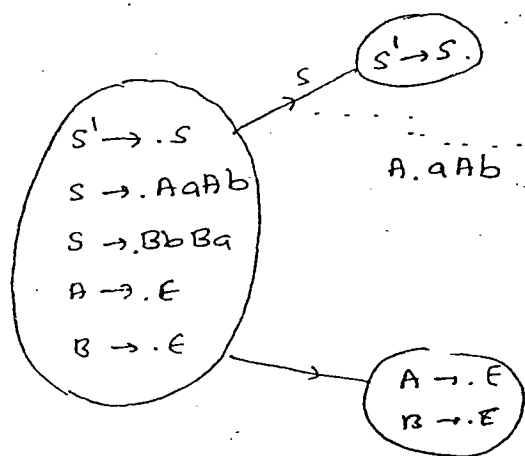
$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

$fol(A) = \{a, b\}$ $fol(A) \cap fol(B) \neq \emptyset$

$fol(B) = \{b, a\}$

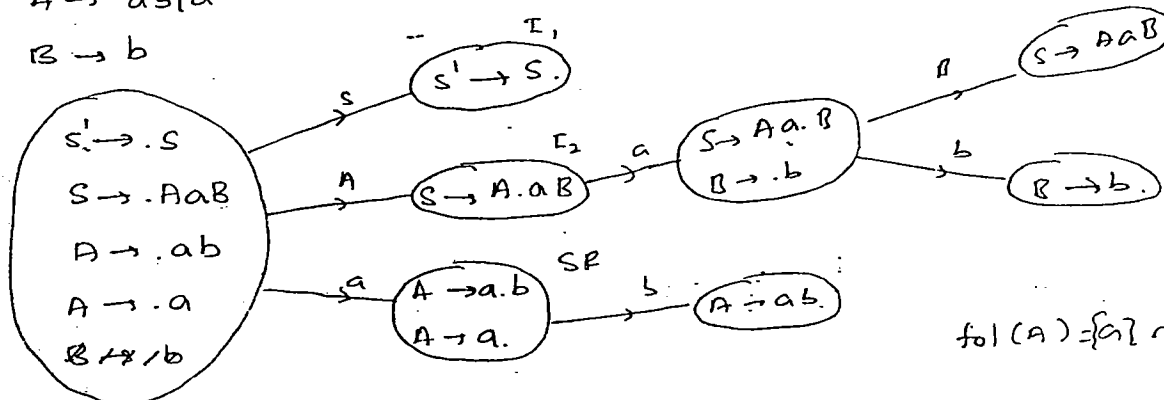
not LR(0) and SLR(1).



3. $S \rightarrow AaB$

$A \rightarrow ab|a$

$B \rightarrow b$

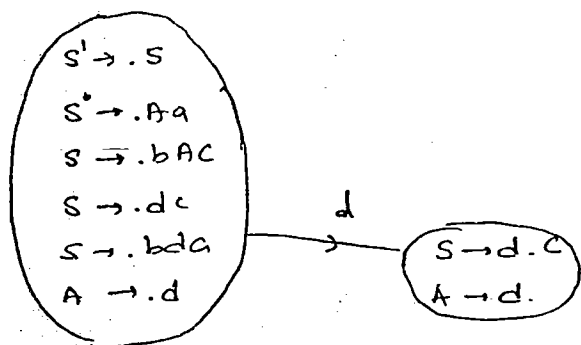


$fol(A) = \{a\} \cap \{b\} = \emptyset$

It is SLR(1) but not LR(0).

4. $S \rightarrow Aa | bAc | dc | bda$

$A \rightarrow d$



$fol(A) = \{a, c\} \cap \{c\} \neq \emptyset$

not LR(0)

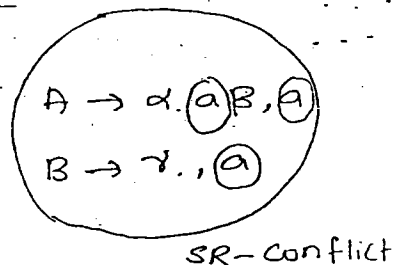
$\Rightarrow$ Not SLR(1)

Goto (i, ∞) .

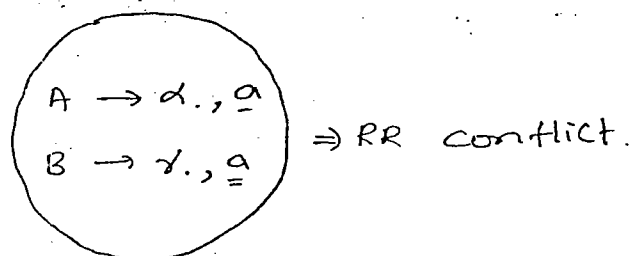
1. while finding the transition lookahead part remain same.
2. while applying the closure lookahead part may change.

Conflicts of in LR(1):

1. SR-Conflict:



2. RR-Conflict:



LR(1) grammar

The grammar for which LR(1) parser is constructed is LR(1) or CLR(1) grammar.

The grammar is LR(1) iff its parse table is free

from multiple entries.

Notes

1. For every grammar SLR(1) grammar is constructed then CLR(1) parser is also constructed but if CLR(1) parser is constructed, we may or may not construct SLR(1).
2. CLR(1) parser is more powerful than SLR(1).
3. No. of entries in SLR(1) parse table $\leq$ (less than or equal to) no. of entries in CLR(1) parse table.

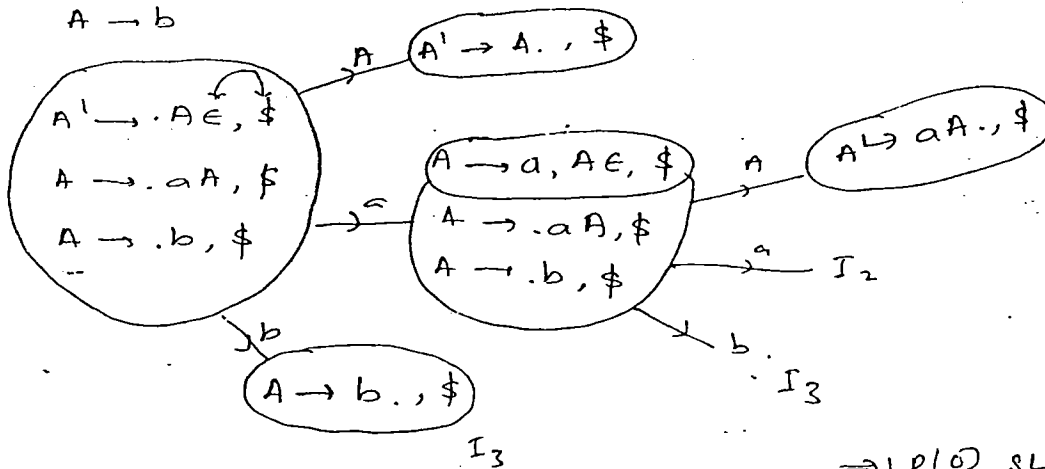
ex: $D \rightarrow Aa.$
 $A \rightarrow b.$

$A \rightarrow aA.$
 $A \rightarrow b$

26

$A' \rightarrow \cdot A\epsilon, \$$
 $A \rightarrow \cdot Aa,$
 $A \rightarrow \cdot b,$

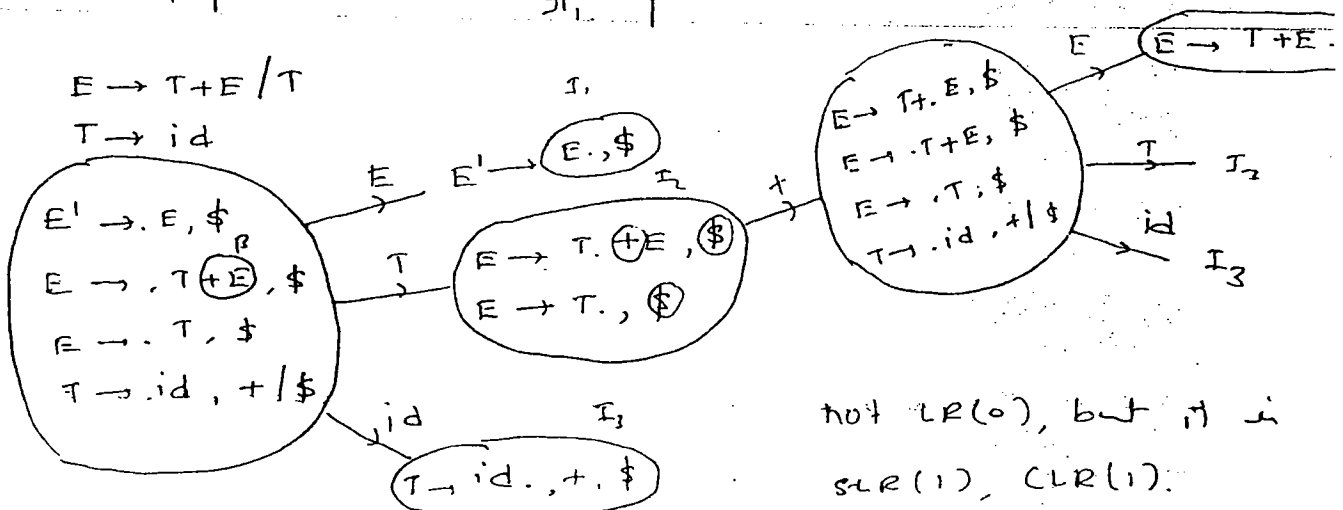
① $A \rightarrow aA$
 $A \rightarrow b$



	a	b	\$	
I_0	S_2	S_3		1
I_1	Acc	Acc	Acc	
I_2	S_2	S_3		
I_3			acc	4
I_4			$\$$	

$\Rightarrow LR(0), SLR(1), CLR(1)$

② $E \rightarrow T+E / T$
 $T \rightarrow id$

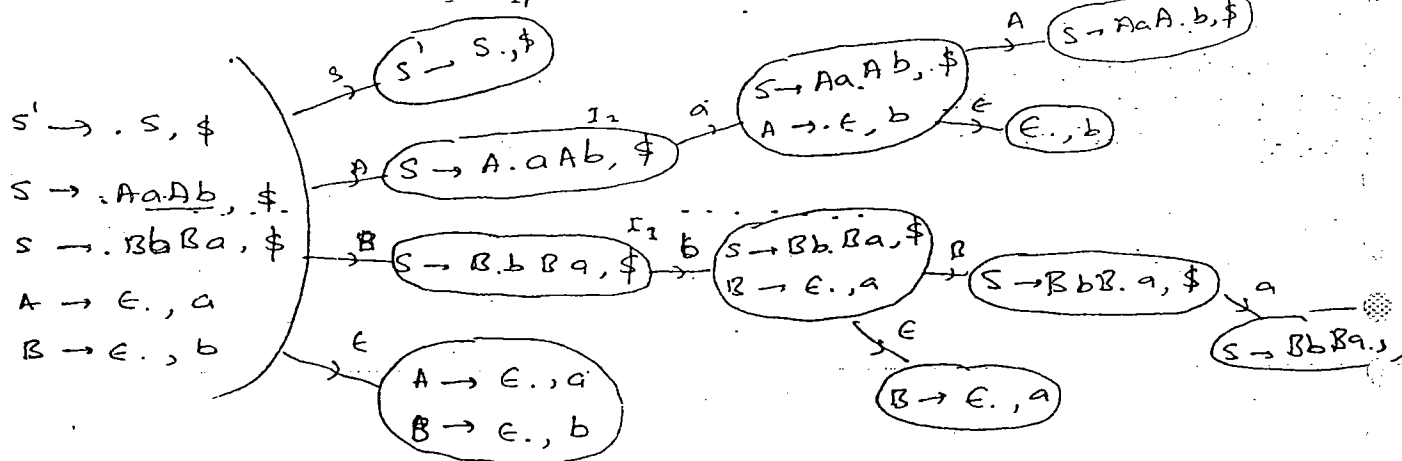


not LR(0), but it is
 SLR(1), CLR(1).
 Not LLR(1)

3. $S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$A \rightarrow \epsilon$



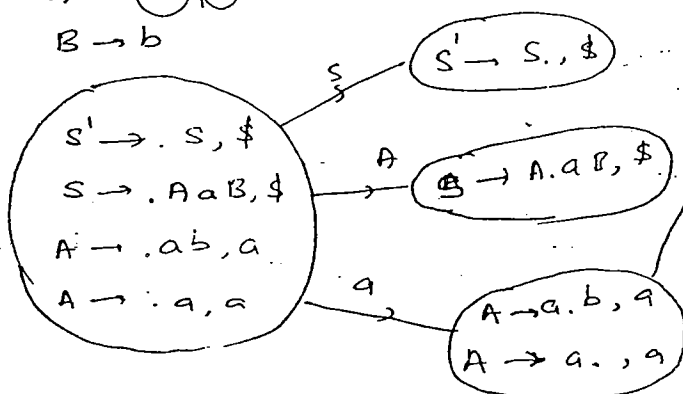
LL(1), CLR(0)

but not LR(0), SLR(1)

4. $S \rightarrow AaB$ Not LRL(1)

$A \rightarrow ab \mid a$

$B \rightarrow b$



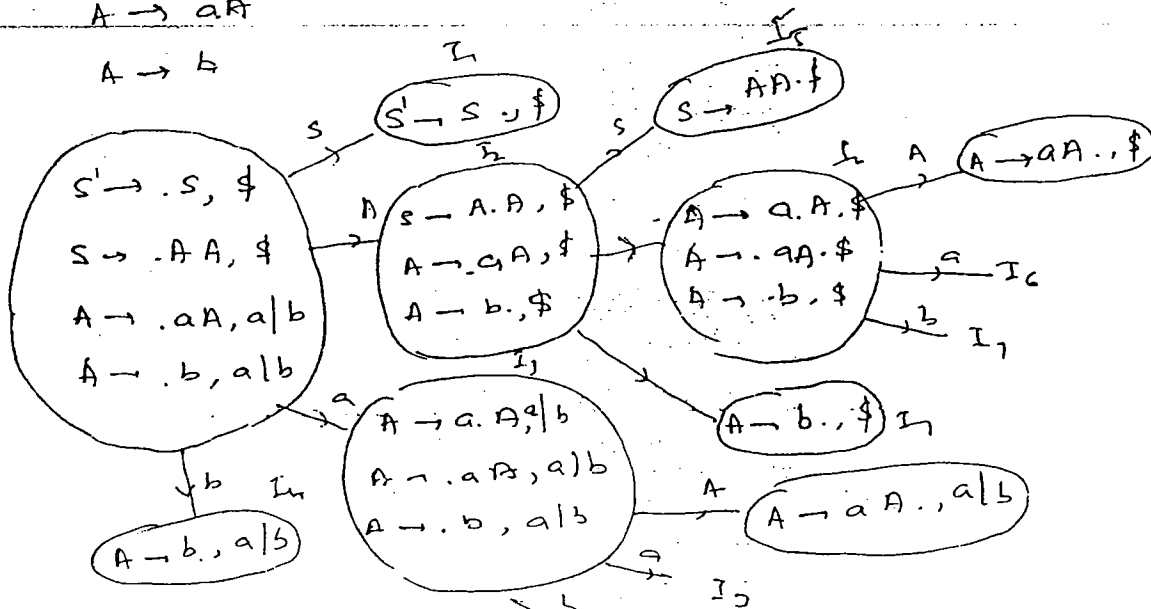
SR conflict in LR(0), but in SLR(1), CLR(1).

$\Rightarrow$ It is not LR(0), but SLR(1), CLR(1).

5. $S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$



	Action			goto	
	a	b	\$	s	D
I_0	S_2	r_4		1	2
I_1	Acc	Acc	Acc		
I_2	S_6	S_7			5
I_3	S_3	S_4			8
I_4	r_3	r_3			
I_5			r_1		
I_6	S_6	S_7			9
I_7			r_3		
I_8	r_2	r_2			
I_9			r_2		

I_4, I_7
 $A \rightarrow b., a|b|f$ I_{47}

$I_3 + I_6$
 $A \rightarrow a.A, a|b|f$ I_{36}
 $A \rightarrow .aA, a|b|f$
 $A \rightarrow .b., a|b|f$

I_8, I_9
 $A \rightarrow aA., a|b|f$ I_{89}

LALR(1) parser

The DFA of CLR(1) parser contain some states with common production part differs by Lookahead part.

Now combine the states with common production part and different lookahead part into a single state and construct the parse table. If the parse table is free from multiple entries then i.e. free from both LR & RR conflict then the grammar is LALR(1) grammar.

Note:

1. If the DFA of CLR(1) parser does not contain more than 1 state with common production part and with different follow part then the grammar is CLR(1) and LALR(1).
2. If the DFA of CLR(1) contains more than one state

part then it may or may not be LALR(1) parser.

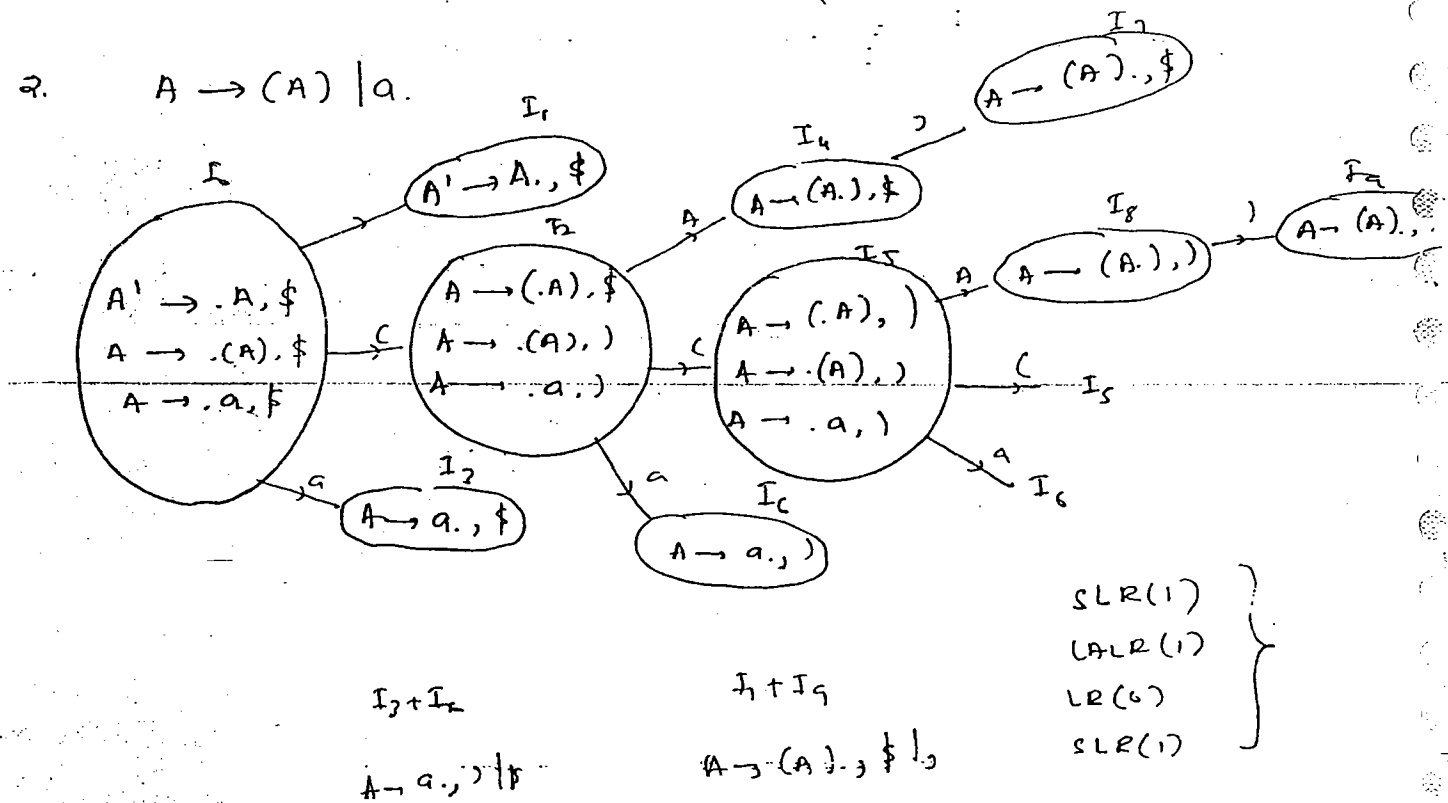
Conflicts in LALR(1)

1. If there is SR conflict in CLR(1) then there is no SR conflict in LALR(1) also.
2. If there is no RR conflict in CLR(1) then there may be RR conflict in LALR(1):

	a	b	\$	S	A
I_0	S_{36}	S_{47}		1	2
I_1	ACC	ACC	ACC		
I_2	S_{36}	S_{47}			5
I_{36}	S_{36}	S_{47}			47
I_{47}	r_3	r_3	r_3		
I_5			$\$$		
I_{89}	r_2	r_2	r_2		

25/12/2010.

2. $A \rightarrow (A) | a$.



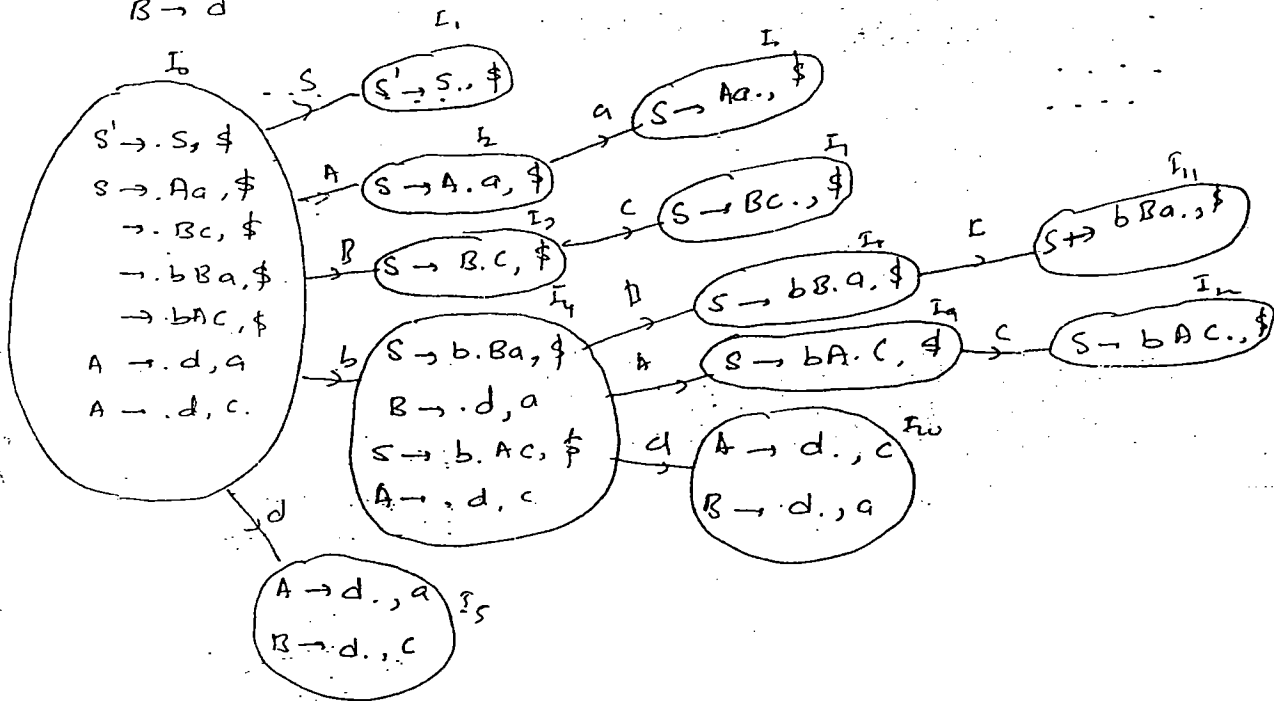
3. $S \rightarrow Aa$
 $S \rightarrow bAc$
 $S \rightarrow Bc$
 $S \rightarrow bBa$
 $A \rightarrow d$
 $B \rightarrow d$

$I_0 + F_0$

$A \rightarrow d., a|c$
 $A \rightarrow d., a|c$

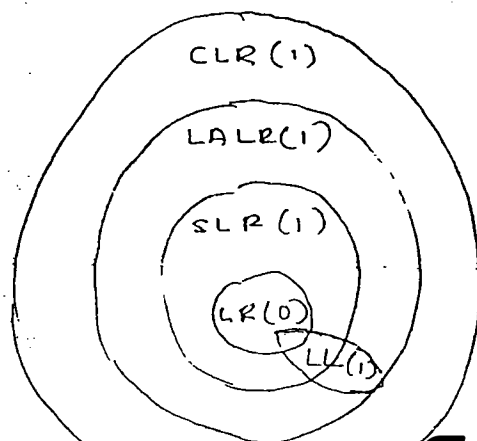
RR conflict

It is CLR(1) 28
 Not LALR(1)
 Not LL(1)
 Not LR(0)
 Not SLR(1)



Note:

1. For a grammar LALR(1) parser is constructed the CLR(1) parser can be constructed.
2. If CLR(1) grammar parser can be constructed for grammar G , then we may or may not construct LALR parser for same grammar.
3. Every LALR(1) grammar is a CLR(1) but every CLR(1) grammar need not be LALR(1).



4. The no. of entries in the LALR(1) parse table is $\leq$ no. of entries in the CLR(1) parse table.
5. No. of entries in LALR(1) parse table = No. of entries in SLR(1) parse table and all the entries are identical for both the parsers.
6. LALR(1) is a medium type of parser and is not powerful as much as a CLR(1) but slightly more powerful than SLR(1) parser. Hence LALR(1) lies b/w CLR(1) & SLR(1).
7. CLR(1) parser is more efficient, powerful and costly among all the parsers.

Problems

1. The grammar $S \rightarrow CC$
 $C \rightarrow aC$
 $C \rightarrow b$

- a. LL(1)
- b. SLR(1) but not LL(1)
- c. LALR(1) but not SLR(1)
- d. CLR(1) but not LALR(1).

2. The grammar $S \rightarrow A|a$
 $A \rightarrow a$ is

- a. LL(1).
- b. LR(0) & LL(1)
- c. LR(0), LL(1), SLR(1), LALR(1), LL(1)
- d. None.

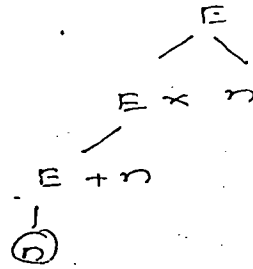
3. $S \rightarrow (s)|a$ and n_1, n_2, n_3 are the entries in the parse tables of SLR(1), LR(1), LALR(1)

- a. $n_1 < n_2 < n_3$
- ☒ b. $n_1 = n_3 \leq n_2$
- c. $n_1 = n_2 = n_3$
- d. $n_1 > n_3 > n_2$

4. $E \rightarrow E+n | E \times n | n$ and $w = n+n \times n$. then... 29

Handle the right sentential form of production.

- a. $n, E+n, E+E \times n$
- b. $n, E+n, E+n \times n$
- c. $n, n+n, n+n \times n$
- d. $n, E+n, E \times n$



5. For SLR(1) and LALR(1) which of the following is true.

- a. Goto entries are different.
- b. Shift entries are identical.
- c. Reduce entries are different.
- d. Error entries are different.

Operator precedence parser

- 1. Constructed for both ambiguous and unambiguous grammars.
- 2. Constructed for only operator grammars.
- 3. Takes the grammar with less complexity.
- 4. Can not be constructed for every grammar.
- 5. Generally used for the language which are mostly useful in scientific application.

Operator Grammar:

The grammar that does not contain E operators or adjacent non-terminals on RHS of any rule is known as operator grammar.

Ex: $E \rightarrow EAE | id$
 $A \rightarrow + | * | -$ } Not operator Grammar.

Ex: $E \rightarrow E+E$
 $\rightarrow E-E$
 $\rightarrow E * E$
 $\rightarrow id$ } Operator Grammar

The operator grammar is a operator precedence grammar if its relational parse table is free from multi entries.

Procedure to construct the precedence parse table:

Let θ_1 and θ_2 be the two operators.

1. If θ_1 has higher precedence than θ_2 , then

$$\theta_1 > \theta_2 \quad \text{or} \quad \theta_2 < \theta_1$$

Again if θ_1 and θ_2 are of equal precedence

$$\theta_1 = \theta_2$$

2. If $A \rightarrow \theta_1 B \theta_2$ is a production where B is variable or ϵ

then, $\theta_1 = \theta_2$

3. If θ_1 and θ_2 are of equal precedence and

a. are of left Associative

then, $\theta_1 > \theta_2$ or $\theta_2 < \theta_1$

b. are of right Associative

then, $\theta_1 < \theta_2$ or $\theta_2 > \theta_1$

ex

$$E \rightarrow E + T / T$$

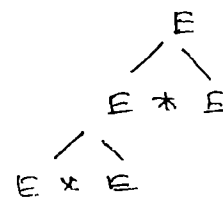
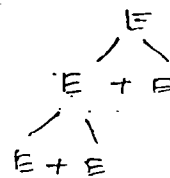
$$T \rightarrow T * F / F$$

$$T \rightarrow id$$

$\downarrow$ θ_1	+	*	id	$\$$
+	>	<	>	>
*	>	>	>	>
id	>	>		>
$\$$	<	<	<	=

$$2. E \rightarrow E + E / E * E / id$$

	*	+	id	\$
*	<>			
+		<>		
id				
\$				



Parse table contains multiple entries

$\therefore$ The grammar is not operator precedence grammar

Note:

Every operator precedence grammar is an operator grammar but every operator grammar need not be operator precedence grammar.

Syntax Directed Translation (SDT)

Semantic Analyser verify the each and every sentence of the source code.

Syntax Analyser or parser takes care of the operators working on required number of operands or not, It does not consider the type of operand.

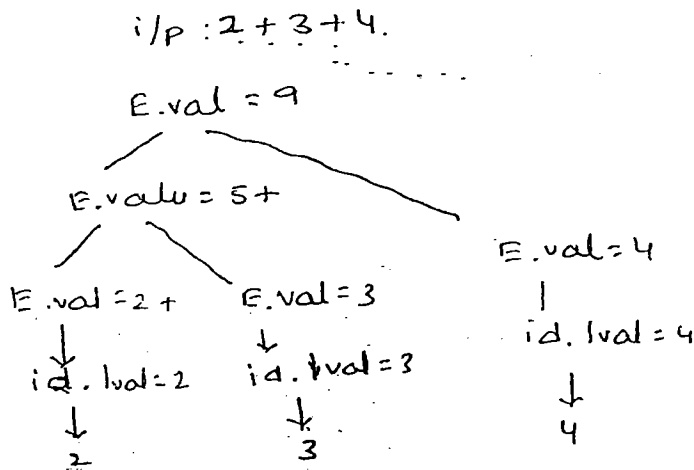
Semantic analysis can be implemented by parsing along with the semantic rules by attaching the semantic rule for each and every grammar rule.

Semantic analysis, the parsing is also verified.

Def SDT:

The grammar with syntactic rules is known as SD

ex:

$$\left. \begin{array}{l} E \rightarrow id \ E + E \quad \{ E.val = E.val + E.val \} \\ E \rightarrow id \quad \{ E.val = id.lval \} \end{array} \right\} \text{SDT.}$$


Note:

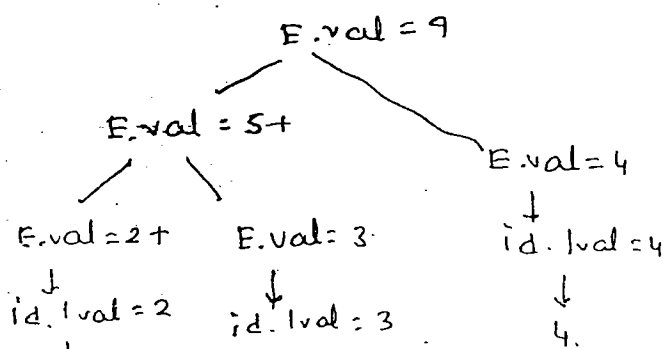
Apart from parsing the SDT can be used

1. To store the type information into symbol table
2. To build the syntax tree or DAG
3. To verify the consistency check like type checking, parameter checking etc
4. to generate intermediate code.
5. To generate the target code.
6. To evaluate the algebraic expression

Annotated Parse Tree (Decorated parse tree)

The parse tree shows the attribute values at each and every node is known as Annotated parse tree.

ex: i/p : 2 + 3 + 4



Types of attributes

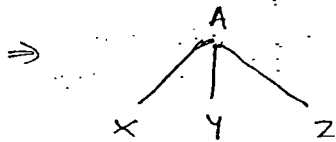
Based on the process of evaluation attributes can be classified into 2 type. 3/

- ① Synthesized Attribute.
- ② Inherited Attribute.

Synthesized Attribute:

The attribute whose value is evaluated in terms of attribute value of its children is called a synthesized attribute.

ex: $A \rightarrow x y z$



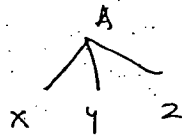
$$A.s = f(x.s \mid y.s \mid z.s)$$

$\downarrow$
Synthesized Attribute.

Inherited Attribute:

Attribute whose value is evaluated in terms of attribute value of its parent or siblings is called Inherited Attribute.

ex: $A \rightarrow x y z$



$$y.s = f(x.s \mid y.s \mid z.s)$$

$\downarrow$
Inherited Attribute.

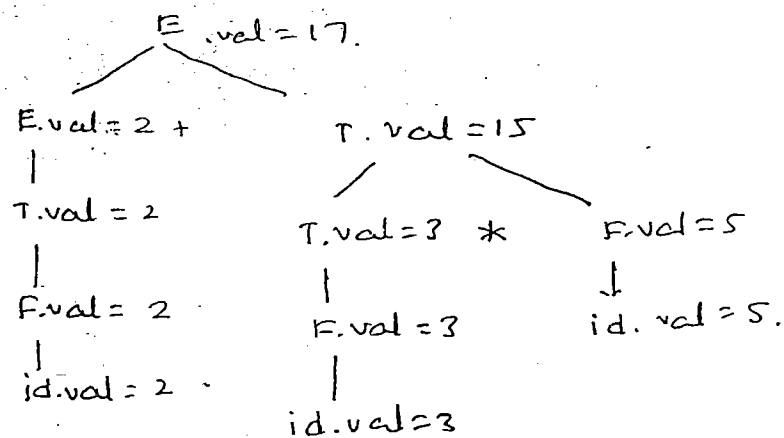
Rules to construct the SDT.

1. Define the grammar by looking at input string
2. construct the parse tree
3. attach the semantic rules by look at required o/p

ex: SDT for the evaluation of an expression:

i/p: $2 + 3 * 5$
o/p: 17

Grammar:

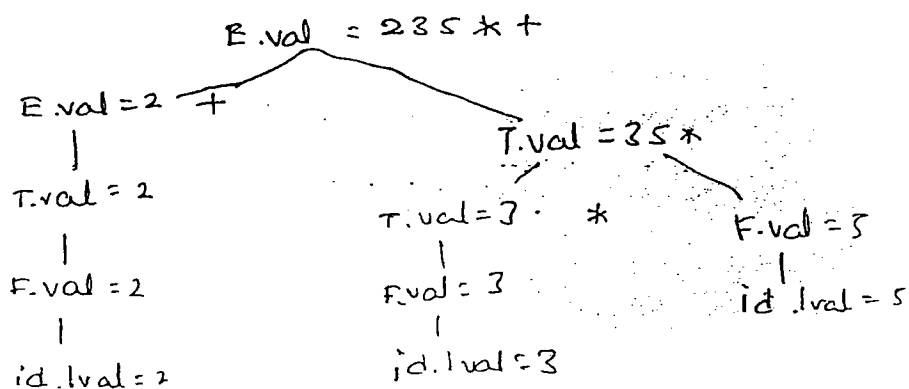
$$\begin{aligned} E &\rightarrow E + T & \{E.val = E.val + T.val\} \\ &| T & \{E.val = T.val\} \\ T &\rightarrow T * F & \{T.val = T.val * F.val\} \\ &| F & \{T.val = F.val\} \\ F &\rightarrow id & \{F.val = id.lval\} \end{aligned}$$


ex 2: SDT to conversion of infix to postfix.

i/p: $2 + 3 * 5$

o/p: $235 * +$

Grammar:

$$\begin{aligned} E &\rightarrow E + T & \{E.val = E.val || T.val || '+'\} \\ &| T & \{E.val = T.val\} \\ T &\rightarrow T * F & \{E.val = T.val || F.val || '*'\} \\ &| F & \{T.val = F.val\} \\ F &\rightarrow id & \{F.val = id.lval\} \end{aligned}$$


SPT to count the parenthesis:

Grammes :

$$S \rightarrow \epsilon \quad \{s.\text{count} = 0\}$$

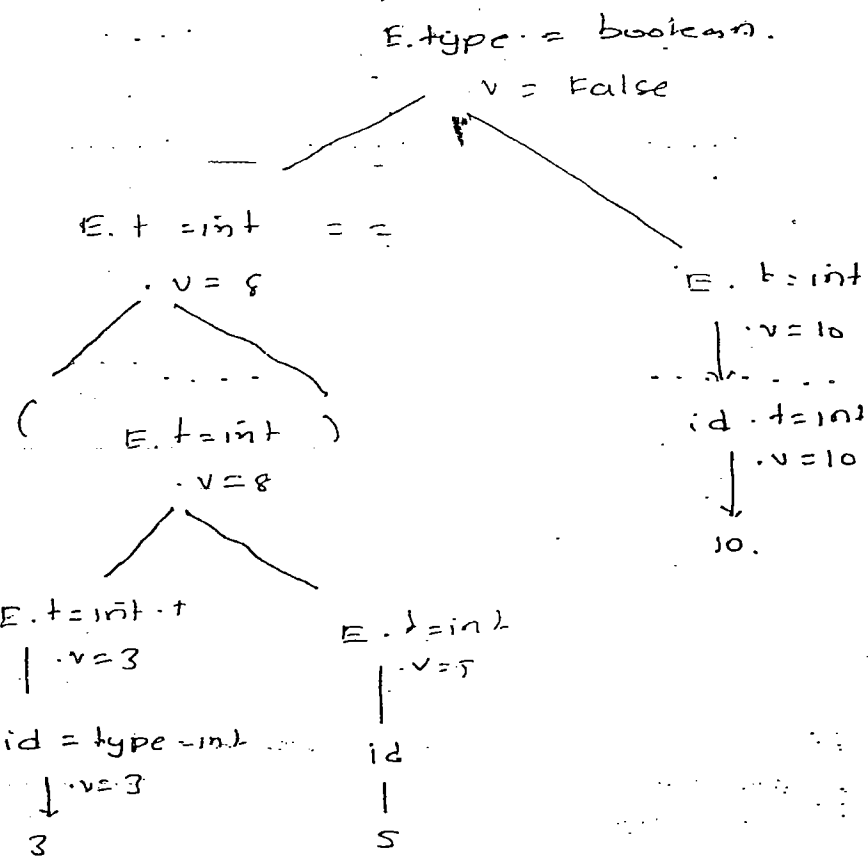
```

graph TD
    A["S.Count = 3"] --- B["C"]
    A --- C["S.Count = 2"]
    A --- D["C"]
    C --- E["C"]
    C --- F["S.Count = 1"]
    C --- G["C"]
    F --- H["C"]
    F --- I["S.Count = 0"]
    F --- J["C"]
    I --- K["C"]
  
```

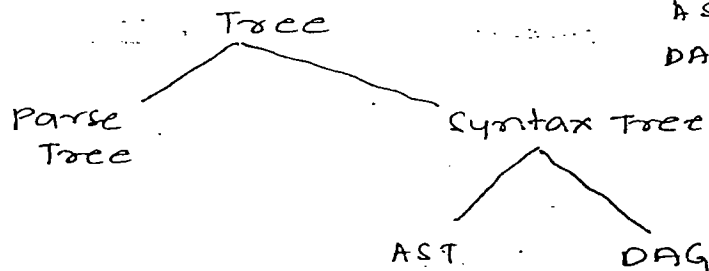
SDT is used for typechecking of boolean and integer data types.

o/p: False.

$E \rightarrow E == E$ { $E_1.type == E_2.type$ } & &
 $\rightarrow (E)$ { $E_1.type == int$ }
 $\rightarrow E + E$ { $E_1.type == E_2.type$ }
 $\rightarrow E * E$ { $E_1.type == E_2.type$ } & &
 $\rightarrow E / E$ { $E_1.type == E_2.type$ } & &
 $\rightarrow E \% E$ { $E_1.type == E_2.type$ } & &
 $\rightarrow id = int$ { $E.type == id.type == int$ }
 $\rightarrow True$ { $E.type == Boolean$ }
 $\rightarrow False$ { $E.type == Boolean$ }



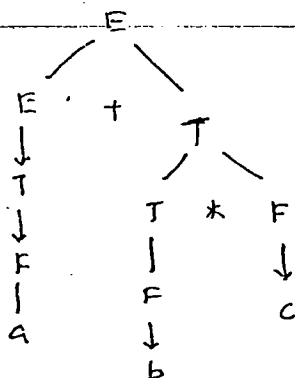
SDT to build Syntax Trees



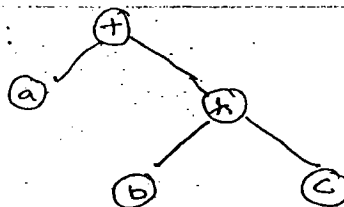
AST - Abstract Syntax Tree
DAG - Directed Acyclic Graph

AST:

$a + b * c$



o/p:



AST:

33

$E \rightarrow E + T \quad \{ E.pbr = \text{m/c node } (E.pbr, +, T.pbr); \}$

$| T \quad \{ E.pbr = T.pbr \}$

$T \rightarrow T * F \quad \{ T.pbr = \text{m/c node } (T.pbr, *, F.pbr); \}$

$| F \quad \{ T.pbr = F.pbr \}$

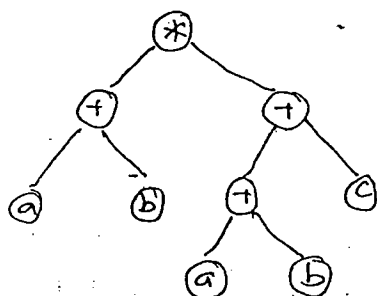
$F \rightarrow id \quad \{ F.pbr = \text{m/c node } (NULL, id, NULL); \}$

DAG

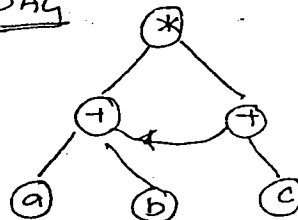
DAG is used to eliminate the common sub expressions. The procedure to construct the SDT for DAG is similar to AST except one restriction. i.e if any node is already created make use of that node instead of going for new creation of same node.

ex: $(a+b) * (a+b+c)$

AST



DAG



Types of SDT

SDT can be defined in two ways

1. S-attributed
2. L-attributed

S-Attribute

1. Uses only synthesized attributes
2. Semantic rules can be placed at the right end of the
3. Attributes are evaluated during Bottom up Parsing

ex: $E \rightarrow FT \quad \{ pf(\#) \}$

$F \rightarrow T * F \quad \{ pf(*) \}$

$\rightarrow a \quad \{ \}$

$T \rightarrow id \quad \{ \}$

This document is available free of charge on



Downloaded by Personal Work (personalwork671@gmail.com)

L-attributed:

1. Uses synthesized and inherited attributes
2. Inherited attribute values are inherited from the parent or left siblings.
3. Semantic rules can be placed on the RHS.
4. Attributes are evaluated using Depth first Right to left process (DFS L to R).

Ex:

$$\begin{aligned} S &\rightarrow A \{ \text{print}(\#) \} B \\ A &\rightarrow +B \{ \text{print}(@) \} C | \epsilon \\ B &\rightarrow aC \{ \text{print}(\dagger) \} | \epsilon \\ C &\rightarrow b \{ \text{print}(\ast) \} \end{aligned}$$

Note:

1. Every S-attributed is L-attributed.
2. Every L-attributed can be converted into S-attributed.

L-attributed

$$\begin{aligned} E &\rightarrow TE' \dagger \\ E' &\rightarrow +T \textcircled{1} E' | \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow \ast F \textcircled{2} T' | \epsilon \\ F &\rightarrow \text{id} \textcircled{3} \end{aligned}$$

S-attributed

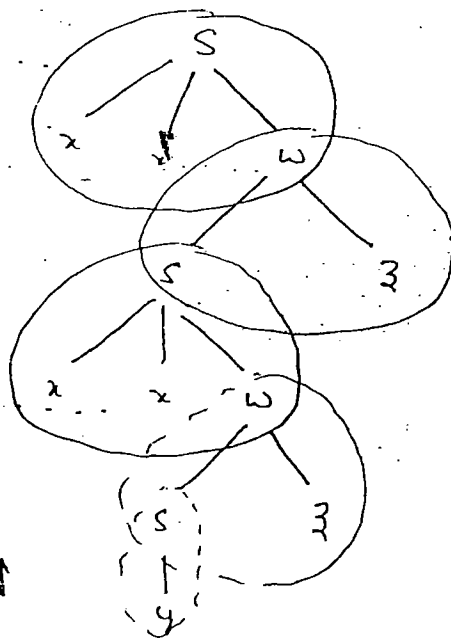
$$\begin{aligned} E &\rightarrow TE' : \\ E' &\rightarrow +TE' | \epsilon \\ M &\rightarrow \epsilon \textcircled{1} \\ T &\rightarrow FT' \\ T' &\rightarrow \ast FRT' | \epsilon \\ R &\rightarrow \epsilon \textcircled{2} \\ F &\rightarrow \text{id} \textcircled{3} \end{aligned}$$

attributes are evaluated
only from children

① $S \rightarrow xny \{ \text{print}(1) \}$
 $y \{ \text{print}(2) \}$
 $n \rightarrow sz \{ \text{print}(3) \}$

If the sdt carries the string $w = xxxxyzz$

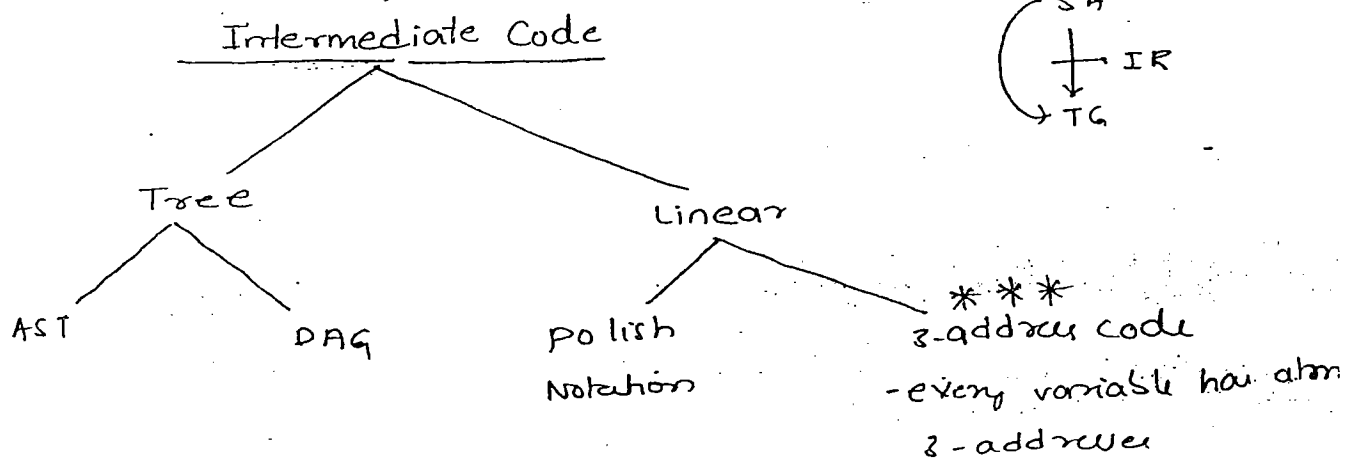
a. 23131 2. 23133 3. 21313



23131 34

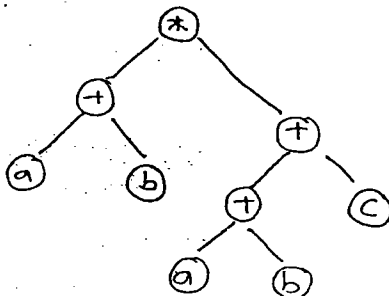
2. $S \rightarrow BC \quad \{ B.x = C.x \}$

- a. S-attributed
- b. L-attributed
- ~~c. Both~~
- d. None

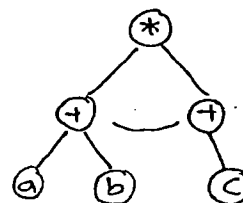


Ex: $(a+b) * (a+b+c)$

1. AST:



2. DAG



3. Polish notation

$ab+ab+c+*$

*** 4.

3-Address code

$$t_1 = a + b$$

$$t_2 = a + b$$

$$t_3 = t_2 + c$$

$$t_4 = t_1 * t_3$$

Types of 2-address code statements

- ① $x = y \text{ op } z$
- ② $x = \text{operator } y. \quad (x = -y)$
- ③ $x = y$
- ④ $\text{goto } L \text{ (unconditional jump)}$
- ⑤ $\text{If relation op } \text{goto } L \text{ (conditional jump)}$
- ⑥ $x = a[i]$
- ⑦ $a[i] = y$
- ⑧ $x = *p$
- ⑨ $x = \$y$
- ⑩ $f(x_1, x_2, x_3, \dots, x_n)$ - not 3 address code

Parameter $\left. \begin{array}{l} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{array} \right\} \text{ 3-Address code.}$

$$y = f_n(1, n)$$

Implementation of 3-Address code:

3-Address code can be represented in 3-ways -

- ① Quadruples.
- ② Triples
- ③ Indirect Triples

Quadruples:

$$x = a + b * c$$

$$\left. \begin{array}{l} t_1 \quad b \quad c \quad * \\ t_2 \quad a \quad t_1 \quad + \\ x \quad - \quad t_2 \quad = \end{array} \right\}$$

$t_1 \quad t_2$ are temporary variables

Advantage:

statements can be moved around for later optimization purpose.

Disadv: Wastage of memory because of temporary variables

Triples:

$$x = a + b * c$$

$$(1) \quad b \quad c \quad *$$

$$(2) \quad a \quad (1) \quad +$$

$$(3) \quad x \quad (2) \quad =$$

Adv: No wastage of memory.

Disadv: Statements cannot be moved around.

Indirect Triples:

Triples are separated in execution order & use it pointers when they are being called.

$$(11) \quad \text{or} \quad (101)$$

$$(12) \quad (102)$$

$$(13) \quad (103)$$

Note:

1. Every part of the source code will be converted into intermediate code.

2. Intermediate code can be generated for declaration, Conditional & control statements, ARRAYS & procedures.

Write the 3-address code for the following.

① $(a+b) * c \mid (d+e)$

$$t_1 = a + b$$

$$t_2 = t_1 * c$$

$$t_3 = d + e$$

$$t_4 = t_2 \mid t_3$$

② $a * b + c \mid d$

$$t_1 = a * b$$

$$t_2 = c \mid d$$

$$t_3 = t_1 + t_2$$

③ `int x = 10;`

`int y = 20;`

`L: SOP(x+y);`

⇒

(1) `x = 10;`

(2) `y = 20`

(3) `goto 2`

(4) `L: SOP(x+y)`

④ `int x = 10`

`int y = 20`

`if (x > y)`

`L1: SOP("TOC");`

`else`

`L2: SOP("<D");`

⇒

(1) `x = 10`

(2) `y = 20`

(3) `if x < y goto L2(6)`

(4) `goto L1(5)`

(5) `L1: SOP("TOC");`

(6) `L2: SOP("<D")`

⑤* `for (int i=0; i<10; i++)`
`{`
`x = i;`
`a[i] = x;`
`}`

⇒ (1) `i=0`

(2) `if i > 10 goto last`

(3) `x = i`

(4) `a[i] = x`

(5) `t = i+1`

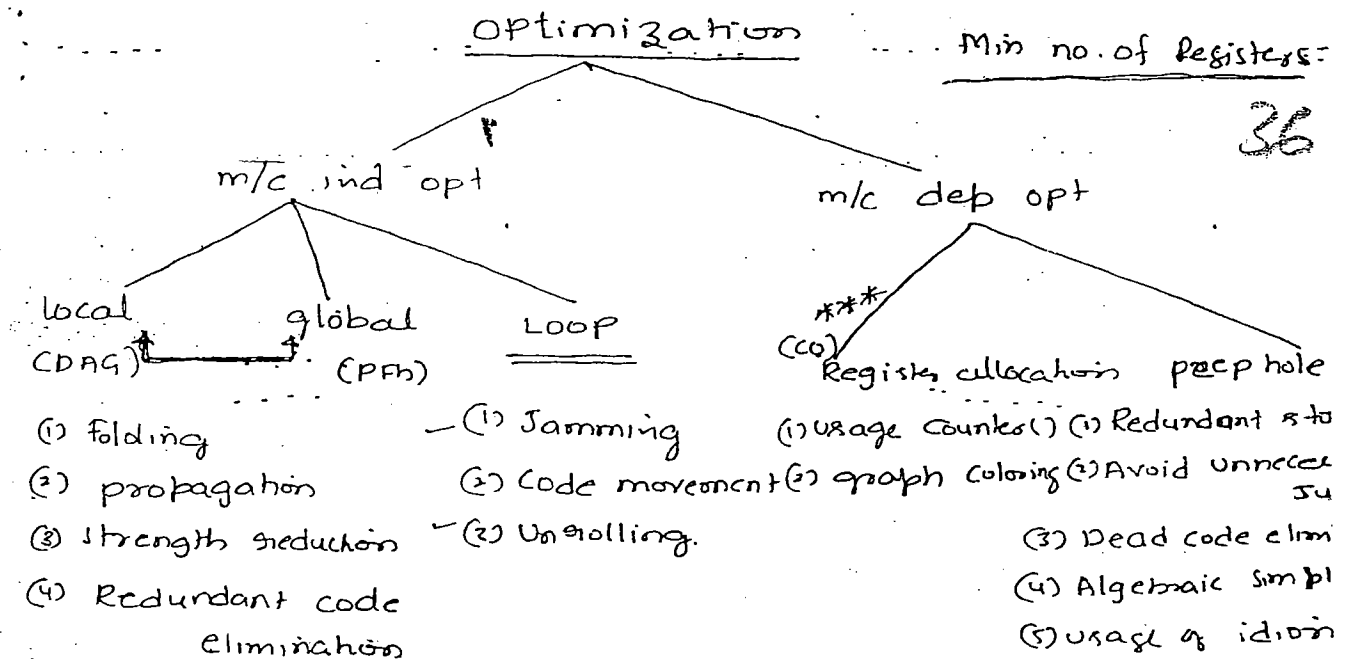
(6) `i = t`

(7) `goto (2)`

(8) `L: last`

Note:

1. Intermediate representation platform independence.
2. Intermediate representation improve the performance of code generation process.



The process of reducing the no. of instructions to improve the performance of the compiler without affecting the outcome of source program is known as optimization.

Optimization of two types

1. machine independent opt or language dependent opt.
2. machine dependent opt or language independent opt.

Machine independent optimization

optimization of 3-address code is known as machine independent optimization.

ex: $x = a + b * c$

$t_1 = b * c$

$t_2 = a + t_1$

$x = t_2$

opt:-

$t_1 = b * c$

$x = a + t_1$

Types of machine independent optimization

1. Local
2. global
3. Loop

Local: The optimization which is performed within a block is known as local optimization, where block is a collection of 3-address statements.

Global: The optimization which is performed at program level is known as Global optimization.

The complete source program is divided into blocks with the help of leaders.

Construct the program flow graph (PFG) for taking every block as a node accordingly, data flow analysis connects the nodes. PFG will help the optimization at program level.

- DAG is used to optimize at block level.
- Blocks are identified with the help of leaders.

Basic block: The collection of 3-address code statements from a leader to next leader without including the next leader is known as Basic block.

Rule to identify the leaders:

1. Convert the source code to 3-address code.
2. First statement is a leader.
3. The statement follows conditional or unconditional jump is a leader.
4. The target of conditional or unconditional jump is a leader.

ex: fact(n)

{ int i = 1;

int f = 1;

for (i = 1; i <= x; i++)

f = f * i;

x = f

}

3-Address code
B₁ { (1) i = 1

(2) f = 1

B₂ (3) if i > x goto 9

(4) f = f * i

(5) x = f

(6) t = i + 1

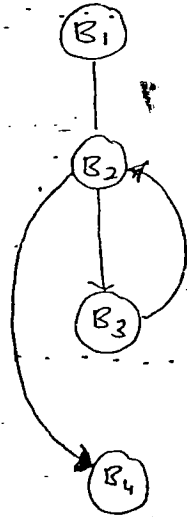
(7) i = t

(8) goto (3)

B₄ (9) L: calling program

leaders: 1, 4, 9, ...

B₃



Characteristics of Local or Global optimization

1. Constant folding
2. Constant propagation
3. Strength reduction
4. Redundant code elimination

Constant folding:

Replacing the value of expression before compile time is known as constant folding.

$$\text{ex: } y = a + b + 3 * 4$$

$$\rightarrow y = a + b + 12.$$

Constant propagation:

Replacing the value of constant before compile time is known as constant propagation.

$$\text{ex: } \pi = 3.142$$

$$x = \frac{\pi}{180} \Rightarrow x = \frac{3.142}{180}$$

Strength Reduction

Replacing the costly operator by cheaper operator is known as strength reduction.

$$\text{ex: } y = 3 * x$$

Redundant

Avoiding the evaluation of any expression more than once.

$$x = a + b$$

$$y = b + a$$

machine

The process of optimizing the Target Code is known as machine dependent optimization.

Types of m/c dependent optimization

1. Register allocation
2. peephole optimization

Register Allocation

It is the process of finding the min. no. of registers that are allocated during the target code optimization. The min no. of registers can be calculated using two algorithms.

- ① Usage Counter
- ② Graph coloring

Peephole optimization:

Characteristics:

1. Redundant store/load instruction.

MOV R0, a

mov a, R0

2. Avoid unnecessary jumps.

1) if $x > y$ goto L₁

2) L₁: goto L₂

3) L₂: goto L₃

4) L₃: goto L₁₀

⇒ if $x > y$ goto L₁₀

3. Dead code elimination

38

#define x 0

if (x)

{

...

...

}

- never executed

- Dead code

4. Algebraic simplification

ex: $y = x + 0$

$\Rightarrow y = x$

ex: $y = x * 1$ ($x \neq 0$)

$\Rightarrow y = x$

5. Usage of idioms

$y = x + 2;$

$i = i + 1; \rightarrow$

$y = x + 2; \}$

$inc(i)$

Loop optimization

The process of optimization with in the loop is known as loop optimization.

characteristic

① Loop Jamming:

Combining the bodies of two loops when ever they are sharing same index variable and no. of iterations.

ex: $\text{for (int } i=0; i \leq 10; i++)$ $\Rightarrow$ $\text{for (int } i=0; i \leq 10; i++)$

$\left(\begin{array}{l} \text{for (int } j=0; j \leq 20; j++) \{ \\ \quad a[i, j] = "100"; \\ \text{for (int } i=0; i \leq 10; i++) \} \\ \quad b[i] = "01"; \end{array} \right. \Rightarrow \left. \begin{array}{l} \text{for (int } j=0; j \leq 20; j++) \\ \quad a[i, j] = "100"; \\ \text{for (int } i=0; i \leq 10; i++) \\ \quad b[i] = "01"; \end{array} \right.$

Code movement:

```
int a = 10; int b = 20;
while (i ≤ 500)
{
    a = i + a * b;
    i++;
}
```

⇒

```
int a = 10;
int b = 20;
int t = a * b;
while (i ≤ 500)
{
    a = i + t;
    i++;
}
```

Loop Unrolling:

```
int i = 0;
while (i ≤ 500)
{
    sop(i);
    i++;
}
```

⇒

```
int i = 0;
while (i ≤ 500)
{
    sop(i);
    i++;
    sop(i);
    i = i + 1;
}
```

0	2	1
1	3	5

reducing no. of iterations.

chaitu0213@y.co.in

9502687942

9849209842